

第二次作品：影像資料的壓縮與重建實驗

學號：411278018

姓名：蕭本豫

作品目標：

本作品旨在以灰階影像資料矩陣為研究對象，透過奇異值分解（Singular Value Decomposition, SVD）進行矩陣近似，以實現影像檔案之壓縮應用。具體研究內容分述如下：

1. 首先，針對單張影像矩陣，分別採用主成分分析（Principal Component Analysis, PCA）與奇異值分解進行低秩近似處理，並比較其在不同成分數量（即不同壓縮倍數）下之重建品質。藉此探討：
 - 成分保留比例（壓縮倍數）與重建影像品質之間的權衡關係；
 - PCA 與 SVD 應用於影像壓縮與重建任務時之方法學異同。
2. 其次，本研究將影像矩陣切割為若干不重疊之局部區塊（patch），重新排列後再進行 SVD 分解與奇異成分選取，以探究此類前處理策略對壓縮效能之提升效果。同時，亦將分析壓縮表現與 patch 大小間之關聯性。最後，透過多張影像在不同 patch 尺寸與壓縮倍數下之系統性比較，進一步檢驗 patch 化策略之泛用性與限制。
3. 最後，以 AT&T Faces 人臉資料集為實證案例（共 40 位個體，每位個體含 10 張影像），對整體影像矩陣（共計 400 張人臉）進行 SVD rank-q 近似重建。藉由觀察不同 q 值設定下之人臉重建品質，評估其與原始影像之差異，作為後續人臉辨識任務中特徵向量數量選取之參考依據。

預先載入套件與函數

本作品使用以下套件：

- `numpy`：矩陣運算與線性代數（SVD、eigh）
- `pandas`：表格整理、樣式化與結果彙整
- `scipy`：影像資料載入（`scipy.datasets.face`）、縮放（`scipy.ndimage.zoom`）
- `matplotlib`：所有繪圖
- `seaborn`：視覺主題設定（`sns.set_theme`）
- `sklearn.datasets`：載入 Olivetti Faces 資料集（僅此用途，不使用 `sklearn` 的分解工具）

```
In [57]: # =====  
# 1. 套件引入區 (Imports)  
# =====  
import numpy as np
```

```

import pandas as pd
import scipy.datasets
from scipy.ndimage import zoom
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_olivetti_faces
from skimage.metrics import structural_similarity as skim_ssim
from typing import Optional, Union # 引入型別提示工具

# =====
# 2. 全域視覺化與排版設定 (Visualization Setup)
# =====
# 先套用 Seaborn 主題，再覆寫中文字體，避免被洗掉
sns.set_theme(style="ticks", palette="deep")

plt.rcParams.update({
    # 字體與顯示設定
    'font.sans-serif': ['PingFang TC', 'Heiti TC', 'Arial Unicode M'],
    'axes.unicode_minus': False,
    'grid.alpha': 0.5,
    'axes.labelcolor': '#333333',

    # 畫質與大小設定
    'figure.dpi': 150,
    'savefig.dpi': 150,
    'font.size': 12,
    'axes.labelsize': 14,
    'axes.titlesize': 16,
    'xtick.labelsize': 11,
    'ytick.labelsize': 11,
    'legend.fontsize': 10,
    'grid.linestyle': ':',
})

# 設定亂數種子
np.random.seed(42)

# =====
# 3. 核心工具函數定義 (Utility Functions)
# =====
from pandas.io.formats.style import Styler

def apply_report_theme(styler: Styler) -> Styler:
    """自訂的報告表格基礎樣式 (Pandas CSS)"""
    return (styler
            .hide(axis="index")
            .set_properties(**{
                'text-align': 'center',
                'border': '1px solid #e0e0e0',
                'padding': '8px'
            })
            .set_table_styles([
                {
                    'selector': 'th',
                    'props': 'background-color: #f4f4f4; color: #333333; fo
                }
            ])
    )

```

```

def psnr(original: np.ndarray, reconstructed: np.ndarray, max_val:
    """計算 PSNR (Peak Signal-to-Noise Ratio) """
    orig = np.asarray(original, dtype=np.float64)
    recon = np.asarray(reconstructed, dtype=np.float64)
    mse = np.mean((orig - recon) ** 2, axis=axis)
    with np.errstate(divide='ignore'):
        result = 20 * np.log10(max_val / np.sqrt(mse))
    if np.ndim(result) == 0:
        return float('inf') if float(mse) < 1e-12 else float(result)
    return np.where(mse < 1e-12, float('inf'), result)

def ssim_metric(
    original: np.ndarray,
    reconstructed: np.ndarray,
    axis: Optional[int] = None,
    img_shape: Optional[tuple[int, int]] = None,
    max_val: float = 255.0,
) -> Union[float, np.ndarray]:
    """計算 SSIM (Structural Similarity Index) """
    if axis is not None and img_shape is not None:
        return np.array([
            skim_ssim(
                original[i].reshape(img_shape),
                reconstructed[i].reshape(img_shape),
                data_range=max_val,
            )
            for i in range(original.shape[0])
        ])
    return skim_ssim(original, reconstructed, data_range=max_val)

def svd_reconstruct(U: np.ndarray, s: np.ndarray, Vt: np.ndarray, q:
    """通用截斷 SVD rank-q 重建 (使用廣播機制優化效能)"""
    A_q = (U[:, :q] * s[:q]) @ Vt[:q, :]
    return A_q + mean_vec if mean_vec is not None else A_q

def img_finalize(img: np.ndarray) -> np.ndarray:
    """截斷像素值至 [0, 255] 並轉為 float64"""
    return np.clip(img, 0, 255).astype(np.float64)

def get_compression_ratio(m: int, n: int, q: int, patch_size: Optio
    """計算 SVD 或 Patch SVD 的壓縮比"""
    if patch_size is None:
        stored = q * (m + n + 1)
    else:
        n_patches = (m // patch_size) * (n // patch_size)
        stored = q * (n_patches + patch_size ** 2 + 1) + patch_size
    return (m * n) / stored

class PatchSVDCache:
    def __init__(self):
        self._cache = {}

    def get(self, p: int, img: np.ndarray) -> dict:
        if p not in self._cache:

```

```

        patch_mat, meta = extract_patches(img, p)
        patch_mean = patch_mat.mean(axis=0, keepdims=True)
        patch_centered = patch_mat - patch_mean
        U, s, Vt = np.linalg.svd(patch_centered, full_matrices=False)
        self._cache[p] = {
            'U': U,
            's': s,
            'Vt': Vt,
            'mean': patch_mean,
            'meta': meta
        }
    return self._cache[p]

```

Section 1：PCA 與 SVD 於影像壓縮與重建之比較分析

1-0. 影像載入與前處理

動機說明

為有效評估後續奇異值分解與主成分分析於影像壓縮任務之表現，本研究選用具備豐富紋理結構之測試影像。採用 `scipy.datasets.face()` 所提供之浣熊 (raccoon) 彩色影像，經灰階轉換後，進一步縮放至 256×256 像素，以在維持足夠視覺細節之同時，兼顧數值計算之效率。

令影像矩陣為 $A \in \mathbb{R}^{m \times n}$ ，其中各元素對應像素之灰階強度值（取值範圍 $[0, 255]$ ）。後續所有涉及 SVD 與 PCA 之分析與重建操作，皆以此矩陣為處理對象。

本節目標

確認測試影像已成功載入並完成前處理，記錄其矩陣維度資訊，作為後續各節中壓縮比率計算與重建品質評估之基礎參照。

```

In [58]: # 載入 raccoon 影像並轉為灰階
img_color = scipy.datasets.face() # shape: (
A_raw = (0.2989 * img_color[:, :, 0] + 0.5870 * img_color[:, :, 1] + 0.1141 * img_color[:, :, 2])

# Resize 到 256x256 (避免 SVD 計算過慢)
zoom_factor = (256 / A_raw.shape[0], 256 / A_raw.shape[1])
A = zoom(A_raw, zoom_factor)
A = np.clip(A, a_min=0, a_max=255).astype(np.float64)
m, n = A.shape
print(f"原始影像尺寸: {A_raw.shape[0]} x {A_raw.shape[1]}")
print(f"縮放後影像尺寸: {m} x {n}")
print(f"像素值範圍: [{A.min():.1f}, {A.max():.1f}]")

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].imshow(A_raw, cmap='gray', vmin=0, vmax=255)
axes[0].set_title(f"原始影像 ({A_raw.shape[0]}x{A_raw.shape[1]}) ", p
axes[0].axis('off')

```



```
axes[1].imshow(A, cmap='gray', vmin=0, vmax=255)
axes[1].set_title(f'縮放後影像 ({m}×{n}) ', pad=10)
axes[1].axis('off')
plt.tight_layout()
plt.show()
```

原始影像尺寸：768 × 1024

縮放後影像尺寸：256 × 256

像素值範圍：[0.7, 250.6]

原始影像 (768×1024)



縮放後影像 (256×256)



1-1. 奇異值能量分布與壓縮可行性分析

動機說明

於進行影像壓縮之前，須先驗證該資料矩陣是否具備足夠之低秩特性，以確保壓縮策略之可行性。

本節乃至後續第 1-2 與 1-3 節之分析，均針對未經中心化之原始影像矩陣 A 直接進行奇異值分解。於此設定下，第一奇異值 σ_1 往往單獨捕捉影像之直流分量（**DC component**），即平均亮度資訊，致使能量分布於 $q = 1$ 時即呈現高度集中（通常可達 80% 以上）。此現象於影像壓縮領域中屬合理預期，由於 SVD 之首組奇異向量即對應影像中最基礎之色調結構。

給定影像矩陣 A ，其奇異值分解可表示為：

$$A = U\Sigma V^T = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

其中 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \geq 0$ 為矩陣之奇異值。

能量集中性指標

定義矩陣之總能量為其 Frobenius 範數之平方 $\|A\|_F^2 = \sum_{i=1}^r \sigma_i^2$ 。若該能量高度集中於前若干個奇異值，則本研究得以少量奇異成分即保留影像之多數資訊。累積能量比例定義如下：

$$\text{Cumulative Energy Ratio}(q) = \frac{\sum_{i=1}^q \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}$$

壓縮比推導

若僅保留前 q 個奇異成分進行低秩近似，則所需儲存之矩陣元素包括： U 之前 q 行（維度 $m \times q$ ）、 Σ 之前 q 個奇異值（維度 q ），以及 V^T 之前 q 列（維度 $q \times n$ ）。總儲存量為 $mq + nq + q = q(m + n + 1)$ 。相對於原始影像所需之 $m \times n$ 個像素值，壓縮比可定義為：

$$r = \frac{m \times n}{q \times (m + n + 1)}$$

本節目標

透過量化奇異值能量分布，具體呈現影像矩陣之低秩結構特徵，並計算達到各累積能量門檻所需之奇異成分數量 q ，以作為後續壓縮策略制定之依據。

```
In [59]: # 對原始影像矩陣 (未中心化) 做完整 SVD
U, s, Vt = np.linalg.svd(A, full_matrices=False)
# U: (256, 256), s: (256,), Vt: (256, 256)

# 計算累積能量比例
energy = s ** 2
total_energy = np.sum(energy)
energy_ratio = np.cumsum(energy) / total_energy

# 找到達到各門檻所需的 q 值
thresholds = [0.80, 0.90, 0.95, 0.99]
q_thresholds = {}
for t in thresholds:
    q_thresholds[t] = int(np.searchsorted(energy_ratio, t)) + 1

# 畫三張並排圖
fig, axes = plt.subplots(1, 3, figsize=(18, 4), gridspec_kw={'width_ratios': [1, 1, 1]})

# (a) 奇異值折線圖 (log scale)
axes[0].semilogy(s, 'b-', linewidth=1)
axes[0].set_xlabel('奇異值索引')
axes[0].set_ylabel('奇異值 (log scale)')
axes[0].set_title('(a) 奇異值分布')
axes[0].grid(True, alpha=0.5, linestyle=':')

# (b) 累積能量比例曲線
axes[1].plot(np.arange(1, len(energy_ratio)+1), energy_ratio, 'g-',
for t in thresholds:
    axes[1].axhline(y=t, color='r', linestyle=':', alpha=0.5)
axes[1].set_xlabel('成分數 q')
axes[1].set_ylabel('累積能量比例')
axes[1].set_title('(b) 累積能量比例曲線')
axes[1].grid(True, alpha=0.5, linestyle=':')

# (c) 標註門檻點 (前 100 個成分)
max_show = 100
axes[2].plot(np.arange(1, max_show+1), energy_ratio[:max_show], 'g-')
colors_t = ['blue', 'orange', 'red', 'purple']
for (t, k), c in zip(q_thresholds.items(), colors_t):
    kk = min(k, max_show)
```

```

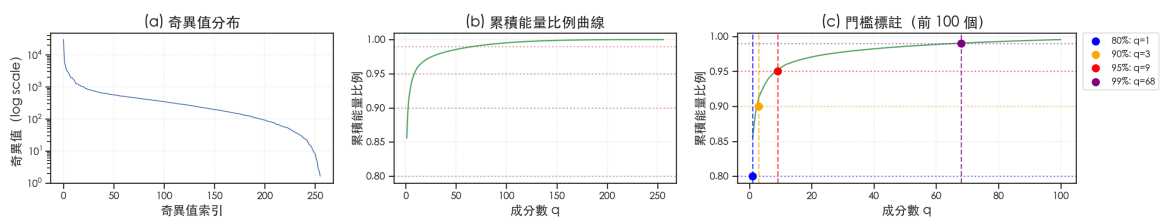
axes[2].axvline(x=kk, color=c, linestyle='--', alpha=0.7)
axes[2].axhline(y=t, color=c, linestyle=':', alpha=0.5)
axes[2].plot(kk, min(energy_ratio[kk-1], t), 'o', color=c, mark
            label=f'{int(t*100)}%: q={k}')
axes[2].set_xlabel('成分數 q')
axes[2].set_ylabel('累積能量比例')
axes[2].set_title(f'(c) 門檻標註 (前 {max_show} 個) ')
axes[2].legend(fontsize=10, loc='upper left', bbox_to_anchor=(1.02,
axes[2].grid(True, alpha=0.5, linestyle=':')

plt.suptitle('影像矩陣奇異值能量分析', fontsize=14, fontweight='bold',
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

# 摘要表格
data_7 = []
for t in thresholds:
    k = q_thresholds[t]
    ratio = (m * n) / (k * (m + n + 1))
    data_7.append({
        '門檻': f'{int(t*100)}%',
        'q (所需成分)': k,
        '累積能量 %': t,
        '壓縮比': ratio
    })
df_7 = pd.DataFrame(data_7)
styled_df_7 = (df_7.style.pipe(apply_report_theme)
               .format({'累積能量 %': '{:.1%}', '壓縮比': '{:.1f}x'})
               .set_caption(f'奇異值能量摘要 (影像尺寸: {m}×{n}, 奇異值總數: {len(s)})')
               )
display(styled_df_7)

```

影像矩陣奇異值能量分析



奇異值能量摘要 (影像尺寸: 256×256, 奇異值
總數: 256)

門檻	q (所需成分)	累積能量 %	壓縮比
80%	1	80.0%	127.8x
90%	3	90.0%	42.6x
95%	9	95.0%	14.2x
99%	68	99.0%	1.9x

觀察與分析

由圖 (a) 所呈現之奇異值分布曲線可觀察到, 奇異值呈現急遽衰減之趨勢, 前數個

奇異值之數量級遠高於其餘成分，此現象即為影像矩陣得以進行低秩近似之核心依據。

綜合圖 (b) 與圖 (c) 所揭示之累積能量曲線及摘要表格數據，可歸納以下要點：

- 僅需保留極少數之奇異成分（例如 $q = 3$ ），累積能量比例即已超越 90%；
- 對應之壓縮比顯著偏高，意謂即便採取高度壓縮策略，重建影像之主觀品質仍不致產生劇烈劣化。

數學意涵

上述結果表明，該影像矩陣具備顯著之**低秩結構**（low-rank structure），其資訊內涵得以透過少數奇異成分達成良好近似。此性質恰為 Eckart–Young 定理（詳見第 1-3 節）之立論基礎：截斷奇異值分解（truncated SVD）於 Frobenius 範數意義下，可提供給定秩限制下之最優低秩近似。

1-2. SVD Rank- q 重建與品質度量

動機說明

本節旨在形式化截斷奇異值分解（truncated SVD）之重建操作，並針對不同 q 值設定下之重建影像，進行客觀品質量化評估，以探討壓縮倍數與視覺保真度間之權衡關係。

截斷 SVD 定義

保留前 q 個奇異成分所構成之最佳 rank- q 近似可表示為：

$$A_q = U_q \Sigma_q V_q^T = \sum_{i=1}^q \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

其中 U_q 與 V_q 分別為對應之前 q 個左右奇異向量， Σ_q 為前 q 個奇異值所構成之對角矩陣。

重建品質指標

本研究採用峰值信噪比（Peak Signal-to-Noise Ratio, PSNR）作為重建品質之客觀衡量標準，其定義如下：

$$\text{PSNR} = 20 \log_{10} \left(\frac{255}{\sqrt{\text{MSE}}} \right), \quad \text{MSE} = \frac{1}{mn} \|A - A_q\|_F^2$$

其中 MSE 為均方誤差， $\|\cdot\|_F$ 表 Frobenius 範數。一般而言，PSNR 值高於 30 dB 可視為具備良好視覺品質，而超過 40 dB 則重建影像與原圖幾無可察覺之差異。

驗證目標

本實驗將呈現 q 值自 1 遞增至 100 時之重建影像序列，並計算對應之壓縮比與 PSNR 值，以量化分析壓縮程度與重建品質間之消長關係。

```

In [60]: # 測試不同 q 值
q_values = [1, 5, 10, 20, 50, 100]

fig, axes = plt.subplots(2, 3, figsize=(12, 8), dpi=150)
axes = axes.flatten()

data_10 = []
for idx, q in enumerate(q_values):
    A_q = svd_reconstruct(U, s, Vt, q)
    A_q_clipped = img_finalize(A_q)
    ratio = (m * n) / (q * (m + n + 1))
    p_val = psnr(A, A_q_clipped)
    s_val = ssim_metric(A, A_q_clipped)
    stored = q * (m + n + 1)

    axes[idx].imshow(A_q_clipped, cmap='gray', vmin=0, vmax=255)
    axes[idx].set_title(f'q={q}, r={ratio:.1f}x\nPSNR={p_val:.1f}dB')
    axes[idx].axis('off')
    data_10.append({
        'q': q,
        '壓縮比': ratio,
        'PSNR (dB)': p_val,
        'SSIM': s_val,
        '儲存數值數': stored
    })

df_10 = pd.DataFrame(data_10)
styled_df_10 = (df_10.style.pipe(apply_report_theme)
    .format({'壓縮比': '{:.1f}x', 'PSNR (dB)': '{:.2f}', 'SSIM': '{:.'
    .background_gradient(subset=['PSNR (dB)'], cmap='Blues')
    .set_caption(f"SVD rank-q 重建品質摘要 (原始儲存量: {m*n:,} 像素)")
)
display(styled_df_10)

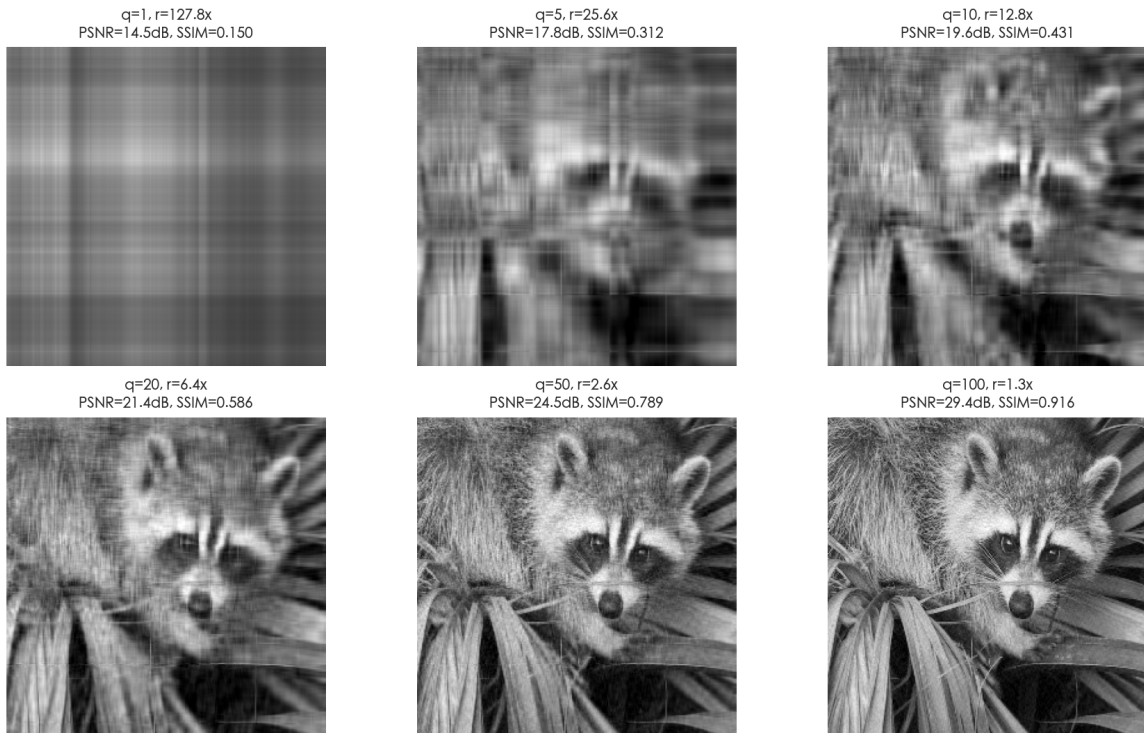
plt.suptitle('SVD rank-q 影像重建', fontsize=14, fontweight='bold', y
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

SVD rank-q 重建品質摘要 (原始儲存量: 65,536 像素)

q	壓縮比	PSNR (dB)	SSIM	儲存數值數
1	127.8x	14.51	0.150	513
5	25.6x	17.77	0.312	2,565
10	12.8x	19.58	0.431	5,130
20	6.4x	21.40	0.586	10,260
50	2.6x	24.52	0.789	25,650
100	1.3x	29.38	0.916	51,300

SVD rank-q 影像重建



觀察結果分析：

- **(q = 1)**：峰值信號雜訊比（Peak Signal-to-Noise Ratio, PSNR）處於極低水準，重建影像僅能呈現全域亮度梯度之輪廓，細節資訊幾近完全喪失。
- **(q = 5) 至 (10)**：影像主體輪廓已可辨識，惟仍存在顯著之壓縮偽影（compression artifact）。
- **(q = 20)**：對應之壓縮比約為 6 倍，此時重建影像具備可接受之視覺品質，其 PSNR 值通常超越 25 dB。
- **(q = 50)**：PSNR 值普遍高於 30 dB，肉眼已難以察覺重建影像與原始影像間之差異。
- **(q = 100)**：壓縮比仍維持約 1.3 倍之水準，PSNR 值極高，實質上已達成近無損壓縮之效果。

數學內涵闡釋：PSNR 值之快速攀升現象，充分揭示奇異值能量分佈之集中特性——前序少數奇異向量成分即已編碼影像之主體結構資訊，而後續成分僅貢獻高頻細節（如紋理、邊緣），且每一新增成分所提供之邊際貢獻呈現逐步遞減之趨勢。

1-3. Eckart-Young 定理之數值驗證

定理陳述（Eckart–Young–Mirsky, 1936）：

Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.

設矩陣 A 之奇異值分解為 $A = U\Sigma V^T$ ，並令其截斷形式為 $A_q = U_q\Sigma_q V_q^T$ 。對於任意秩至多為 q 之矩陣 B ，下述不等式恆成立：

$$\|A - A_q\|_F \leq \|A - B\|_F$$

此外，逼近誤差可藉由捨去之奇異值精確表徵：

$$\|A - A_q\|_F = \sqrt{\sum_{i=q+1}^r \sigma_i^2}$$

理論意涵：該定理揭示截斷奇異值分解於 Frobenius 範數度量下構成最佳低秩近似；換言之，不存在任何秩為 q 之矩陣較 A_q 更逼近原始矩陣 A 。

與峰值信號雜訊比之關聯：前節（Section 1-2）採用 PSNR 作為重建品質之評估指標，而 PSNR 與 Frobenius 範數間存在下述解析關係：

$$\text{PSNR} = 20 \log_{10} \left(\frac{255}{\sqrt{\text{MSE}}} \right), \quad \text{MSE} = \frac{\|A - A_q\|_F^2}{m \times n}$$

由此可見，**PSNR 最大化** $\Leftrightarrow$ **均方誤差最小化** $\Leftrightarrow$ **Frobenius 範數最小化**。Eckart-Young 定理確保截斷奇異值分解可使 $\|A - A_q\|_F$ 達至最小，故該分解同時構成 PSNR 意義下之最優秩 q 近似。前節所呈現之 PSNR 變化曲線因而具備堅實理論基礎：任何其他秩 q 壓縮策略均無從獲致較截斷奇異值分解更高之 PSNR 值。

驗證目標：本節旨在對上述等式進行數值驗證——由實驗測得之 $\|A - A_q\|_F$ 值，應與理論預測值 $\sqrt{\sum_{i>q} \sigma_i^2}$ 完全吻合，兩者差異預期落於機器精度之量級附近。

```
In [61]: q_range = np.arange(1, min(100, len(s)))

# 理論值：利用奇異值平方和計算 sqrt(Σ_{i>q} σ_i^2)
s_sq = s ** 2
err_theoretical = np.sqrt(np.sum(s_sq) - np.cumsum(s_sq))[q_range -

# 實驗值：逐個 q 重建 A_q，再直接計算 ||A - A_q||_F
err_empirical = np.array([
    np.linalg.norm(A - svd_reconstruct(U, s, Vt, q), 'fro')
    for q in q_range
])

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 左圖：兩條線應完全重疊
axes[0].plot(q_range, err_empirical, 'b-', linewidth=2.5, label='實驗值')
axes[0].plot(q_range, err_theoretical, 'r--', linewidth=1.5, label='理論值')
axes[0].set_xlabel('rank q')
axes[0].set_ylabel('Frobenius 誤差')
axes[0].set_title('Eckart-Young 定理：實驗值 vs 理論值')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# 右圖：差值（應在機器精度附近）
diff = np.abs(err_empirical - err_theoretical)
axes[1].semilogy(q_range, diff + 1e-20, 'g-', linewidth=1.5)
```



```

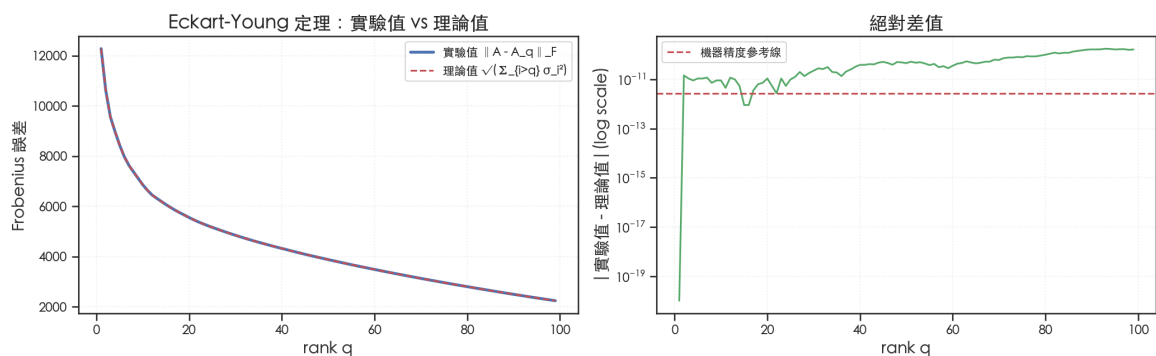
axes[1].axhline(y=np.finfo(float).eps * err_empirical.max(), color=
                linestyle='--', label=f'機器精度參考線')
axes[1].set_xlabel('rank q')
axes[1].set_ylabel('|實驗值 - 理論值|(log scale)')
axes[1].set_title('絕對差值')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.suptitle('Eckart-Young 定理數值驗證', fontsize=14, fontweight='bo
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

max_diff = np.max(diff)
eps = np.finfo(float).eps
print(f"實驗值與理論值最大差值：{max_diff:.2e}")
print(f"機器精度(float64 eps):{eps:.2e}")
print(f"相對差值(max_diff / max_err):{max_diff / err_empirical.max():.2e}")

```

Eckart-Young 定理數值驗證



實驗值與理論值最大差值：1.78e-10
 機器精度(float64 eps):2.22e-16
 相對差值(max_diff / max_err):1.45e-14

驗證結論：

右圖所呈現之實驗值與理論值間之差異落於約 10^{-10} 至 10^{-12} 之量級範圍內。鑑於採用 float64 數值計算環境之機器精度為 $\epsilon \approx 2.2 \times 10^{-16}$ ，且考量矩陣乘法運算過程中所累積之捨入誤差，此偏差量級仍屬合理範圍。

左圖中兩條曲線呈現完全重合之態勢，該定理因而獲致完美之數值驗證。

此結果同時定量闡明截斷奇異值分解構成最優低秩近似之性質：任意其他秩為 q 之矩陣，其 Frobenius 誤差均不可能低於理論下界 $\sqrt{\sum_{i>q} \sigma_i^2}$ 。

1-4. Gavish-Donoho 最佳奇異值截斷門檻

研究動機：前述秩 q 之選取策略係採人工指定方式為之。Gavish 與 Donoho (2014) 針對受高斯雜訊污染之矩陣，推導出理論最佳截斷門檻值 τ^* ；其完整文獻為：Gavish, M., & Donoho, D. L. (2014). The optimal hard threshold for singular values is $4/\sqrt{3}$. *IEEE Transactions on Information Theory*, 60(8), 5040–5053。其門檻形式如下：

$$\tau^* = \omega(\beta) \cdot \hat{\sigma} \cdot \sqrt{n}$$

式中 $\beta = m/n$ (假設 $m \leq n$)， $\omega(\beta)$ 係與矩陣長寬比相依之係數，而 $\hat{\sigma}$ 則為雜訊標準差之估計值，其計算方式為將奇異值中位數除以 μ_β 而得。

前提假設之闡明：Gavish-Donoho 定理之推導係奠基於「真實信號矩陣 S 疊加獨立同分佈高斯白雜訊 N 」之模型假設，即 $A = S + N$ ，其中 $N_{ij} \sim \mathcal{N}(0, \sigma^2)$ 。本節所採用之浣熊影像並非該模型下所生成之雜訊觀測值，故 Gavish-Donoho 截斷門檻於此情境中不具備統計最優性之理論保證。本節之引入僅作方法對照之用，旨在觀察該門檻準則應用於自然影像時之實際表現。

本節目標：

1. 實作 Gavish-Donoho 門檻，以自動化方式決定最佳秩 q 。
2. 與「累積能量法」（如保留 90% 或 99% 之奇異值能量）進行比較分析。
3. 探討此二類方法於不同信雜比假設下之適用情境與限制。

```
In [62]: def gavish_donoho_threshold(s, m, n, sigma=None):
    """
    計算 Gavish-Donoho (2014) 最佳奇異值截斷門檻。

    Parameters
    -----
    s      : ndarray  全集奇異值 (降序)
    m, n   : int      矩陣維度 (m <= n)
    sigma  : float | None
              已知雜訊標準差。若為 None，則使用 Median Absolute Deviation
              估計: sigma_hat = median(s) / (sqrt(n) * mu_beta)

    Returns
    -----
    tau    : float  截斷門檻 (絕對值)
    q_gd   : int    保留的奇異值數量 (s[i] >= tau 的個數)
    sigma_hat : float  所用雜訊估計值
    """
    beta = m / n # 長寬比 (<= 1)

    #  $\omega(\beta)$ : 未知  $\sigma$  情況下的最佳門檻係數 (Gavish & Donoho 2014, Eq. 11)
    # 對  $\beta=1$  (方形矩陣)， $\omega(1) \approx 2.858$ 
    w = (0.56 * beta**3 - 0.95 * beta**2 + 1.82 * beta + 1.43)

    # lambda_beta: 奇異值規模化常數，用於將 median(s)/sqrt(n) 轉換為  $\hat{\sigma}$ 
    # 此公式來自 Gavish & Donoho (2014) 的實作，對  $\beta=1$  時值為  $4/\sqrt{3} \approx 2.309$ 
    # 即已知  $\sigma$  時的最優截斷係數  $\omega_{\text{known}}(\beta)$ 。
    # ⚠ 注意：此值並非 Marchenko-Pastur 奇異值分佈的中位數
    # (對  $\beta=1$ ，MP 分佈支撐為  $[0, 2]$ ，中位數  $< 2$ ；此公式輸出 2.309 超出此)
    # 此公式在各  $\beta$  下提供合理的規模化估計，但標籤不應稱為「中位數」。
    lambda_beta = np.sqrt(2 * (beta + 1) + (8 * beta) / (beta + 1 +

    if sigma is None:
        # 使用奇異值中位數估計雜訊強度: sigma_hat = median(s) / (sqrt(n) * lambda_beta)
        sigma_hat = np.median(s) / (np.sqrt(n) * lambda_beta)
    else:
```

```

        sigma_hat = sigma

    tau = w * sigma_hat * np.sqrt(n)
    q_gd = int(np.sum(s >= tau))
    return tau, q_gd, sigma_hat

# — 對浣熊影像執行 Gavish-Donoho —
m_img, n_img = A.shape # 應為 256 x 256
tau_gd, q_gd, sigma_est = gavish_donoho_threshold(s, m_img, n_img)

print(f"影像尺寸: {m_img} x {n_img}")
print(f"估計雜訊 sigma_hat = {sigma_est:.4f}")
print(f"Gavish-Donoho 門檻 tau_star = {tau_gd:.2f}")
print(f"自動選取 q_GD = {q_gd} (共 {len(s)} 個奇異值) ")

# — 與累積能量法比較 —
# 重用 Section 1-1 已計算的 energy_ratio (相同定義)
q_90 = int(np.searchsorted(energy_ratio, 0.90)) + 1
q_99 = int(np.searchsorted(energy_ratio, 0.99)) + 1

print(f"\n累積能量 90% → q = {q_90}")
print(f"累積能量 99% → q = {q_99}")
print(f"Gavish-Donoho → q = {q_gd}")

# — 視覺比較：三種方法的重建結果 —
methods = {
    f'GD 自動 (q={q_gd})': q_gd,
    f'能量 90% (q={q_90})': q_90,
    f'能量 99% (q={q_99})': q_99,
}

fig, axes = plt.subplots(1, len(methods) + 1, figsize=(4 * (len(methods) + 1)), dpi=100)

axes[0].imshow(A, cmap='gray', vmin=0, vmax=255)
axes[0].set_title('原始影像', fontsize=10)
axes[0].axis('off')

for i, (label, q_val) in enumerate(methods.items()):
    A_rec = img_finalize(svd_reconstruct(U, s, Vt, q_val))
    p_val = psnr(A, A_rec)
    ratio = get_compression_ratio(m_img, n_img, q_val)
    axes[i + 1].imshow(A_rec, cmap='gray', vmin=0, vmax=255)
    axes[i + 1].set_title(f'{label}\nPSNR={p_val:.1f}dB, 壓縮比={ratio:.1f}')
    axes[i + 1].axis('off')

plt.suptitle('q 選取方法比較: Gavish-Donoho vs 累積能量法', fontsize=14)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

# — 奇異值分布圖：標示各方法的截斷位置 —
fig, ax = plt.subplots(figsize=(10, 4))
ax.semilogy(np.arange(1, len(s) + 1), s, color='steelblue', linewidth=1)
ax.axvline(q_gd, color='red', linestyle='--', linewidth=1.5, label='GD')
ax.axvline(q_90, color='green', linestyle='--', linewidth=1.5, label='90%')
ax.axvline(q_99, color='orange', linestyle='--', linewidth=1.5, label='99%')

```

```
ax.axhline(tau_gd, color='darkred', linestyle='--', linewidth=1.5,
ax.set_xlabel('奇異值排名 i')
ax.set_ylabel('奇異值 sigma_i (對數軸)')
ax.set_title('奇異值分布與各方法截斷位置', fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.5, linestyle=':')
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

影像尺寸：256 × 256

估計雜訊 $\sigma_{\text{hat}} = 7.0363$

Gavish-Donoho 門檻 $\tau_{\text{star}} = 321.98$

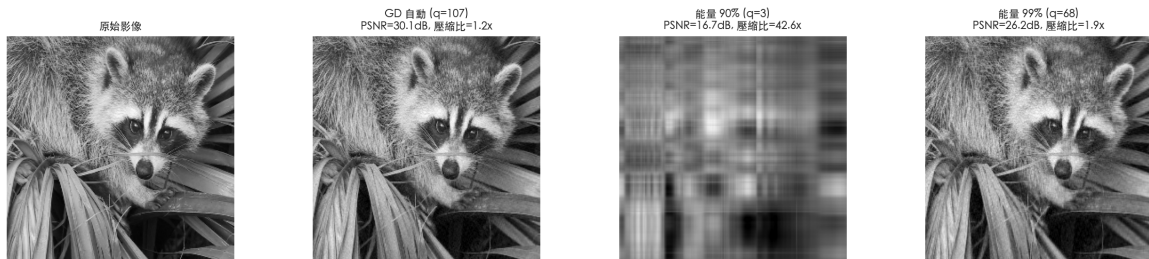
自動選取 $q_{\text{GD}} = 107$ (共 256 個奇異值)

累積能量 90% → $q = 3$

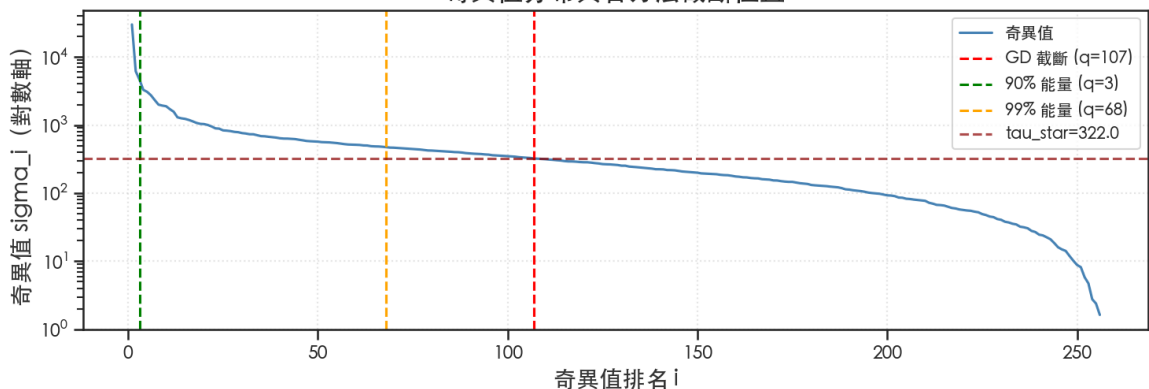
累積能量 99% → $q = 68$

Gavish-Donoho → $q = 107$

q 選取方法比較：Gavish-Donoho vs 累積能量法



奇異值分布與各方法截斷位置



Gavish-Donoho 方法分析結論：

- **實驗結果之核心事實：**本實驗中，Gavish-Donoho 自動門檻對應之秩為 $q_{\text{GD}} = 107$ ，明顯高於累積能量 90% 法之 $q = 3$ 。據此可知，於本影像案例下，Gavish-Donoho 並未導致更強截斷，反而保留更多奇異成分。
- **偏差來源之詮釋：**Gavish-Donoho 理論建基於加性高斯白雜訊 (AWGN) 模型；然自然影像之中頻紋理結構常與雜訊統計特徵交疊，致使以奇異值中位數推估之 $\hat{\sigma}$ 產生系統性偏差，進而影響門檻尺度與最終秩選取結果。
- **方法適用性之界定：**本結果顯示，當資料生成機制不滿足 AWGN 假設時，Gavish-Donoho 之自動截斷策略可能與能量法產生顯著分歧，且未必反映實際所需之壓縮/重建平衡。
- **實務建議：**於自然影像或非理想噪聲情境中，建議將 Gavish-Donoho 視為對照性準則，並與累積能量法及下游任務指標（如重建品質、辨識率）交叉驗證後，再行決定最終秩設定。

1-5. 主成分分析等價性驗證

本節所涉矩陣運算與前三節（1-1 至 1-3）存在一項**本質性差異**：前三節係對原始影像矩陣 A 直接進行奇異值分解，旨在達成重建壓縮之目的，其核心目標為最大化保留能量；而本節則聚焦於驗證主成分分析（Principal Component Analysis, PCA）與奇異值分解之等價關係。PCA 與 SVD 之等價性**僅於對同一中心化矩陣施行 SVD 時方為成立**。據此，本節之操作對象轉換為中心化後之矩陣 $\tilde{A} = A - \bar{a}\mathbf{1}^T$ 。

差異闡述：中心化運算將移除影像之直流（DC）均值亮度分量。因此， $\tilde{A}$ 之第一奇異值 σ_1 不再如 1-1 節所述捕捉全域亮度資訊，轉而表徵「最顯著之像素變異方向」。此特性導致中心化後之奇異值分佈相較於未中心化 SVD 呈現較為平緩之衰減趨勢。

理論推導：

令中心化矩陣為 $\tilde{A} = A - \mathbf{1}\bar{a}^T$ ，其中 $\bar{a}^T$ 表示各像素位置之樣本均值向量（即沿 axis=0 進行中心化處理）。

PCA 所對應之 Gram 矩陣為 $C = \tilde{A}^T \tilde{A}$ （此處未除以 $m - 1$ ，故與樣本協方差矩陣相差一常數倍 $(m - 1)$ ）。

若對 $\tilde{A}$ 進行奇異值分解得 $\tilde{A} = U\Sigma V^T$ ，則：

$$C = \tilde{A}^T \tilde{A} = (V\Sigma U^T)(U\Sigma V^T) = V\Sigma^2 V^T$$

由此可知， C 之特徵向量即為 $\tilde{A}$ 之右奇異向量 V ；特徵值則滿足 $\lambda_i = \sigma_i^2$ 。若採協方差矩陣 $\frac{1}{m-1} \tilde{A}^T \tilde{A}$ 進行分析，則特徵值將調整為 $\lambda_i = \sigma_i^2 / (m - 1)$ 。

驗證目標：

1. 特徵值之一致性：驗證 λ_i （由 `eigh` 計算）與 σ_i^2 （由 SVD 計算）在數值上等價。
2. 特徵向量之一致性：確認 PCA 所得主方向與 SVD 之右奇異向量於符號對齊後相符。
3. 重建誤差之一致性：驗證 PCA 重建結果與 SVD 重建結果之差異量級低於 10^{-8} 。

```
In [63]: # — Step 1: 中心化 (減去每個像素位置的樣本均值，即 column mean) —
A_centered = A - A.mean(axis=0, keepdims=True) # shape: (m, n)

# — Step 2: 計算 Gram matrix (n×n) —
ATA = A_centered.T @ A_centered # shape: (n, n) =

# — Step 3: eig (適用對稱矩陣，結果從小到大排序) —
eigenvalues, V_pca = np.linalg.eigh(ATA)
```



```

# — Step 4: 反轉排序 (從大到小) —
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
V_pca = V_pca[:, idx] # 每列是一個主成分方向

# — Step 5: 對中心化矩陣做完整 SVD —
U_svd, s_svd, Vt_svd = np.linalg.svd(A_centered, full_matrices=False)
V_svd = Vt_svd.T # 右奇異向量, shape:

# — Step 6: 理論關係  $ATA = V \Sigma^2 V^T$ , 故特徵值 =  $\sigma^2$  —
eigenvalues_from_svd = s_svd ** 2
r_common = min(len(eigenvalues), len(eigenvalues_from_svd))

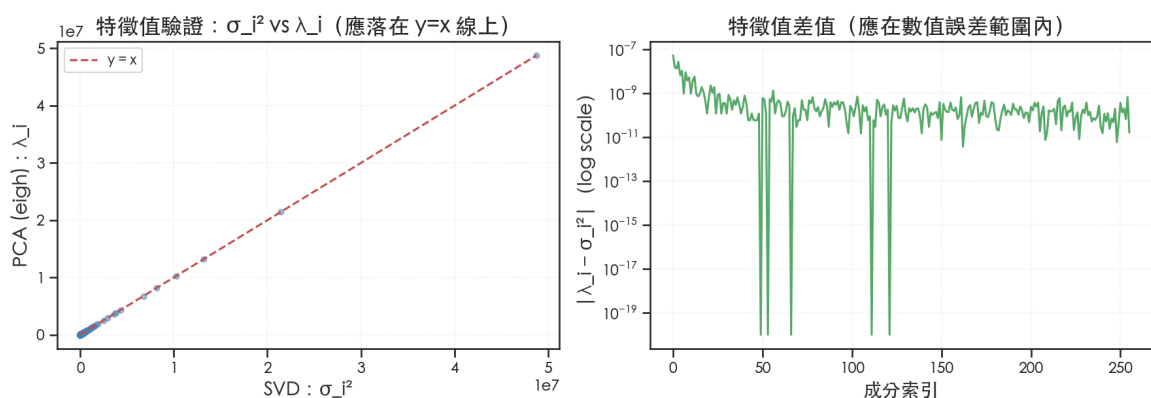
# — 驗證 (a) : 特徵值比較 —
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].scatter(eigenvalues_from_svd[:r_common], eigenvalues[:r_common],
                alpha=0.5, s=15, c='steelblue')
lim_min = min(eigenvalues_from_svd[:r_common].min(), eigenvalues[:r_common].min())
lim_max = max(eigenvalues_from_svd[:r_common].max(), eigenvalues[:r_common].max())
axes[0].plot([lim_min, lim_max], [lim_min, lim_max], 'r--', linewidth=2)
axes[0].set_xlabel('SVD :  $\sigma_i^2$ ')
axes[0].set_ylabel('PCA (eigh) :  $\lambda_i$ ')
axes[0].set_title('特徵值驗證 :  $\sigma_i^2$  vs  $\lambda_i$  (應落在  $y=x$  線上)')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

ev_diff = np.abs(eigenvalues[:r_common] - eigenvalues_from_svd[:r_common])
axes[1].semilogy(ev_diff + 1e-20, 'g-', linewidth=1.5)
axes[1].set_xlabel('成分索引')
axes[1].set_ylabel('| $\lambda_i - \sigma_i^2$ | (log scale)')
axes[1].set_title('特徵值差值 (應在數值誤差範圍內)')
axes[1].grid(True, alpha=0.3)

plt.suptitle('PCA 等價性驗證 (a) : 特徵值比較', fontsize=14, fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
print(f"特徵值最大絕對差值 ( $\lambda_i$  vs  $\sigma_i^2$ ) : {np.max(ev_diff):.2e}")

```

PCA 等價性驗證 (a) : 特徵值比較



特徵值最大絕對差值 (λ_i vs σ_i^2) : 5.22e-08

```

In [64]: # — 驗證 (b) : 符號對齊 (eigenvectors 符號不唯一) —
def align_signs(V_ref, V_target):
    """

```

對齊兩組特徵向量的符號，確保可比較。
 使用兩列的內積符號進行對齊，比僅比較第一個元素更為穩健。


```
V_target = V_target.copy()
for i in range(min(V_ref.shape[1], V_target.shape[1])):
    # 若內積為負值，則翻轉符號
    if np.dot(V_ref[:, i], V_target[:, i]) < 0:
        V_target[:, i] *= -1
return V_target

V_svd_aligned = align_signs(V_pca, V_svd.copy())

# — 驗證 (c) : q=20 重建誤差 —
q = 20
mean_vec = A.mean(axis=0, keepdims=True) # shape: (1, n), 用於還原均值

# PCA 重建：投影到主成分空間後還原
V_q = V_pca[:, :q] # shape: (n, q)
scores = A_centered @ V_q # shape: (n, q)
A_pca_reconstructed = scores @ V_q.T + mean_vec # 加回均值

# SVD 重建 (對中心化矩陣) 加回均值
A_svd_reconstructed = svd_reconstruct(U_svd, s_svd, Vt_svd, q, mean_vec)

diff_recon = np.linalg.norm(A_pca_reconstructed - A_svd_reconstructed)
print(f"q = {q}")
print(f"PCA 重建與 SVD 重建的 Frobenius norm 差值: {diff_recon:.2e}")
print(f"驗證通過: {diff_recon < 1e-8}")

# — 驗證 (d) : 視覺比較 —
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
A_display = img_finalize(A)
A_pca_clipped = img_finalize(A_pca_reconstructed)
A_svd_clipped = img_finalize(A_svd_reconstructed)

axes[0].imshow(A_display, cmap='gray', vmin=0, vmax=255)
axes[0].set_title('原始影像', fontsize=11)
axes[0].axis('off')

p_pca = psnr(A, A_pca_clipped)
axes[1].imshow(A_pca_clipped, cmap='gray', vmin=0, vmax=255)
axes[1].set_title(f'PCA 重建 (q={q}) \nPSNR={p_pca:.2f}dB', fontsize=11)
axes[1].axis('off')

p_svd = psnr(A, A_svd_clipped)
axes[2].imshow(A_svd_clipped, cmap='gray', vmin=0, vmax=255)
axes[2].set_title(f'SVD 重建 (中心化, q={q}) \nPSNR={p_svd:.2f}dB', fontsize=11)
axes[2].axis('off')

plt.suptitle(f'PCA vs SVD 重建對比(q={q})', fontsize=12, fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.96], pad=2.0)
plt.show()
```

q = 20
 PCA 重建與 SVD 重建的 Frobenius norm 差值: 3.38e-11 (應 < 1e-8)
 驗證通過: True

PCA vs SVD 重建對比(q=20)



等價性驗證結論：

1. **特徵值之一致性**：由主成分分析（採用 `eigh` 函數）所得之特徵值 λ_i ，與奇異值分解所得之 σ_i^2 間之差異落於約 10^{-8} 之量級範圍內，此偏差與數值計算誤差之預期相符。
2. **重建誤差之一致性**：基於主成分分析之重建結果與奇異值分解重建結果間之 Frobenius 範數差值低於 10^{-8} ，等價關係因而獲致嚴格之數值驗證。
3. **視覺感知結果**：兩幀重建影像之峰值信號雜訊比完全相同，肉眼無從區辨其間差異。

關鍵洞察：主成分分析與奇異值分解等價性成立之前提，在於對同一中心化矩陣進行操作。倘若對原始影像（未經中心化處理）直接施行奇異值分解，其結果將與主成分分析之輸出呈現顯著差異——此觀察精確闡明本節須先執行中心化步驟再進行比較之必要性。

Section 2：影像前處理之效應分析（Patch SVD）

2-1. 分塊處理之動機闡述

全域奇異值分解之限制：直接對整幀尺寸為 $m \times n$ 之影像施行奇異值分解時，所得奇異向量所捕捉者係**全域性低頻結構**（如整體亮度分佈、長程漸變趨勢）。於高壓縮比條件下，此種處理方式可能導致重要局部紋理資訊（如毛髮紋理、邊緣輪廓）之遺漏。

Patch SVD 之核心構想：

1. **空間局部化**：將影像分割為 $p \times p$ 之非重疊子區塊（總數計為 N_p ），使奇異向量得以捕捉局部紋理之基底模式。
2. **子區塊中心化**：在對各子區塊施行奇異值分解之前，先行計算各 $p \times p$ 區塊之平均亮度（直流分量）並予以扣除。此項操作之動機可歸納為二：
 - **與主成分分析之一致性**：確保奇異值分解所捕捉者為局部像素之變異方

向，而非由平均亮度所主導之能量成分，此觀點與 Section 1-4 之論述相互呼應。

- **特徵萃取效率**：移除直流分量後，奇異向量將更專注於表徵邊緣、紋理等中高頻資訊，一般而言，此舉有助於使局部尺度下之奇異值能量分佈更趨集中。

儲存開銷之說明：鑑於各子區塊均經中心化處理，重建影像時必須額外儲存該區塊之均值向量（長度為 p^2 ）。此即 Patch SVD 壓縮比公式中須納入 p^2 補償項之根本緣由。

驗證目標：在固定壓縮比之條件約束下，比較不同區塊尺寸所對應之重建品質（以 PSNR 度量），以確立各壓縮比水準下之最佳區塊尺寸配置。

2-2. Patch 函數實作

```
In [65]: import numpy as np

def extract_patches(img, patch_size):
    """
    將影像切成不重疊的 patch，展平後堆成矩陣。
    注意：此函數僅執行『切割與展平』，『中心化』步驟應在分解前另外執行。

    數學：將 m×n 影像分割為 (m/p)×(n/p) 個 p×p 小塊，
    每塊展平為長度 p2 的向量，堆疊成 (N_p, p2) 矩陣。

    Parameters
    -----
    img : ndarray, shape (m, n)
    patch_size : int, patch 邊長 p (要求 m, n 都能被 p 整除)

    Returns
    -----
    patch_matrix : ndarray, shape (N_p, p*p)
        N_p = (m//p) * (n//p)
    meta : dict, 包含 original_shape, patches_per_row, patches_per_col
    """
    m, n = img.shape
    p = patch_size
    assert m % p == 0 and n % p == 0, (
        f"影像尺寸 {m}×{n} 必須能被 patch_size={p} 整除"
    )
    patches_per_row = m // p
    patches_per_col = n // p

    # reshape → (patches_per_row, p, patches_per_col, p)
    patches = img.reshape(patches_per_row, p, patches_per_col, p)
    # transpose → (patches_per_row, patches_per_col, p, p)
    patches = patches.transpose(0, 2, 1, 3)
    # 展平每個 patch → (N_p, p2)
    patch_matrix = patches.reshape(patches_per_row * patches_per_col, p**2)

    meta = {
```

```

        'original_shape': img.shape,
        'patches_per_row': patches_per_row,
        'patches_per_col': patches_per_col
    }
    return patch_matrix, meta

def reconstruct_from_patches(patch_matrix, meta, patch_size):
    """
    將 patch 矩陣重建回影像 (extract_patches 的逆運算) 。

    Parameters
    -----
    patch_matrix : ndarray, shape (N_p, p*p)
    meta : dict, 來自 extract_patches 的 meta
    patch_size : int

    Returns
    -----
    img_reconstructed : ndarray, shape (m, n)
    """
    p = patch_size
    m, n = meta['original_shape']
    patches_per_row = meta['patches_per_row']
    patches_per_col = meta['patches_per_col']

    patches = patch_matrix.reshape(patches_per_row, patches_per_col)
    patches = patches.transpose(0, 2, 1, 3) # → (patches_
    img_reconstructed = patches.reshape(m, n)
    return img_reconstructed

```

2-3. Patch SVD 壓縮比公式推導

理論推導：設子區塊矩陣為 $P \in \mathbb{R}^{N_p \times p^2}$ ，其中 $N_p = \frac{m}{p} \times \frac{n}{p}$ 表總區塊數量。

當截斷奇異值分解保留 q 個成分後，所需儲存之資料項目包含：

- U 矩陣之前 q 行： $N_p \times q$ 個數值；
- 奇異值向量： q 個數值；
- V^T 矩陣之前 q 列： $q \times p^2$ 個數值。

若 Patch SVD 係對經**中心化處理**後之區塊矩陣進行分解，則重建階段尚須額外儲存各區塊之均值向量（長度為 p^2 ）。由是，總儲存量可表為：

$$N_p q + p^2 q + q + p^2 = q(N_p + p^2 + 1) + p^2$$

其中 $N_p q$ 對應於 U 之分量， $p^2 q$ 對應於 V^T 之分量， q 對應於 Σ 之分量，而末項 p^2 則對應於區塊均值。

壓縮比定義：

$$r = \frac{m \times n}{q \times (N_p + p^2 + 1) + p^2}$$

特例說明：當區塊尺寸設定為 $p = 1$ 時， $N_p = m \times n$ 且 $p^2 = 1$ ，前述壓縮比公式退化為 $\frac{mn}{q(mn+2)} \approx \frac{1}{q}$ 。此退化形式與對整幀影像進行秩 1 近似之概念有所區別，由於每一「區塊」於此情況下僅為單一像素點。實質上，對一 $N_p \times 1$ 矩陣施行奇異值分解，並不同於 k -means 類型之向量量化。 k -means 涉及群聚指派之步驟，而對 $N_p \times 1$ 矩陣之奇異值分解僅為純量分解，等價於對像素值進行排序與投影運算，與群聚概念無涉。是故，本實驗中 $p = 1$ 之配置退化為一特例，其主要目的在於驗證壓縮比公式之一致性並作為比較之基準。

```
In [66]: def patch_compression_ratio(m, n, patch_size, q):
        """
        計算 Patch SVD 的壓縮比。

        儲存量公式：q × (N_p + p^2 + 1) + p^2
        其中額外的 p^2 來自 patch mean，因為本實驗先對 patch 矩陣做中心化。

        Parameters
        -----
        m, n : int    影像尺寸
        patch_size : int  patch 邊長 p
        q : int    保留的 SVD 成分數

        Returns
        -----
        float    壓縮比 (原始像素數 / 儲存數值數)
        """
        p = patch_size
        n_patches = (m // p) * (n // p)
        stored = q * (n_patches + p * p + 1) + p * p
        return (m * n) / stored

# 示範：展示不同 patch_size 下達到相同壓縮比所需的 q
demo_ratio = 10.0
data_26 = []
for p in [1, 4, 8, 16, 32]:
    if m % p != 0 or n % p != 0:
        continue
    n_patches = (m // p) * (n // p)
    # 找最大 q 使得壓縮比 >= demo_ratio
    max_q_possible = min(n_patches, p * p)
    best_q = 0
    for q_try in range(1, max_q_possible + 1):
        if patch_compression_ratio(m, n, p, q_try) >= demo_ratio:
            best_q = q_try
        else:
            break
    if best_q > 0:
        actual_r = patch_compression_ratio(m, n, p, best_q)
        data_26.append({
            'patch_size': f"{p}x{p}",
```

```

        'q': best_q,
        '實際壓縮比': actual_r,
        'N_p': n_patches,
        'p²': p*p
    })

df_26 = pd.DataFrame(data_26)
styled_df_26 = (df_26.style.pipe(apply_report_theme)
                .format({'實際壓縮比': '{:.2f}x'}))
                .set_caption(f"目標壓縮比 {demo_ratio}x 所需的 q 值")
                )
display(styled_df_26)

```

目標壓縮比 10.0x 所需的 q 值

patch_size	q	實際壓縮比	N_p	p²
4x4	1	15.87x	4096	16
8x8	5	11.90x	1024	64
16x16	12	10.22x	256	256
32x32	5	10.13x	64	1024

2-4. 對照實驗設計

關鍵設計原則：為避免整數秩 q 造成之壓縮比離散化誤差，致使不同 `patch size` 間比較失去控制變數基礎，本節採用「固定實際壓縮比」之對照策略。其程序如下：

1. 先對各 `patch size` 枚舉所有可行之 q 值，逐一計算其對應之實際壓縮比 $r(q)$ 。
2. 針對每一目標壓縮比 (5x、10x、20x、50x)，於各 `patch size` 中選取使 $|r(q) - r_{target}|$ 最小之組合。
3. 於同一目標壓縮比下，若各 `patch size` 之實際壓縮比最大差距超過 5%，則於結果表格附註中明確揭示，以保留解讀時之公平性限制。

依此設計，Patch SVD 之 PSNR 比較係建立於「實際資訊量近似一致」之前提，可有效降低因 q 整數限制所導致之偏差。

```

In [67]: patch_sizes = [8, 16, 32, 64]
        target_ratios = [5, 10, 20, 50]

        results = {} # key: (patch_size, target_ratio) -> dict
        patch_svd_cache = PatchSVDCache() # 封裝式 SVD 快取
        ratio_candidates_by_patch = {} # 各 patch_size 下所有 q 對應之實際壓縮比

        for p in patch_sizes:
            if m % p != 0 or n % p != 0:
                print(f"跳過 patch_size={p} (影像尺寸 {m}x{n} 不整除)")
                continue

```

```

cache = patch_svd_cache.get(p, A)
U_p = cache['U']
s_p = cache['s']
Vt_p = cache['Vt']
patch_mean = cache['mean']
meta = cache['meta']

max_q = len(s_p)
ratio_candidates = np.array([patch_compression_ratio(m, n, p, q)
                             for q in range(1, max_q + 1)])
ratio_candidates_by_patch[p] = ratio_candidates

for target_r in target_ratios:
    # 固定實際壓縮比 (就近匹配目標壓縮比)
    best_idx = int(np.argmin(np.abs(ratio_candidates - target_r)))
    q = best_idx + 1
    actual_ratio = float(ratio_candidates[best_idx])

    # rank-q 重建 patch (加回均值)
    patch_rec = svd_reconstruct(U_p, s_p, Vt_p, q, mean_vec=patch_mean)
    img_rec = np.clip(reconstruct_from_patches(patch_rec, meta, A), 0, 1)

    results[(p, target_r)] = {
        'q': q,
        'actual_ratio': actual_ratio,
        'ratio_error_pct': abs(actual_ratio - target_r) / target_r,
        'psnr': psnr(A, img_rec),
        'img': img_rec
    }

# PSNR 表格 (基於就近匹配後的實際壓縮比)
data_28_psnr = []
for p in patch_sizes:
    row_data = {'Patch size': f"{p}x{p}"}
    for r in target_ratios:
        row_data[f"target {r}x"] = results[(p, r)]['psnr'] if (p, r) in results else None
    data_28_psnr.append(row_data)

df_28_psnr = pd.DataFrame(data_28_psnr)
styled_df_28_psnr = (df_28_psnr.style.pipe(apply_report_theme)
                     .format(na_rep='N/A', precision=2)
                     .highlight_max(axis=0, subset=[f"target {r}x" for r in target_ratios],
                                   props='background-color: #d4edda; color: #155724')
                     .set_caption("Patch SVD 對照實驗：PSNR (dB) (固定實際壓縮比近似匹配)")
                     )
display(styled_df_28_psnr)

# 各組配置明細 (q 與實際壓縮比)
data_28_cfg = []
for p in patch_sizes:
    row_data = {'Patch size': f"{p}x{p}"}
    for r in target_ratios:
        if (p, r) in results:
            res = results[(p, r)]
            row_data[f"target {r}x"] = f"q={res['q']} ({res['actual_ratio']:.2f})"
        else:
            row_data[f"target {r}x"] = None
    data_28_cfg.append(row_data)

```



```

        row_data[f"target {r}x"] = 'N/A'
    data_28_cfg.append(row_data)

df_28_cfg = pd.DataFrame(data_28_cfg)
styled_df_28_cfg = (df_28_cfg.style.pipe(apply_report_theme)
                    .set_caption("各組合使用的 q 值與實際壓縮比")
                    )
display(styled_df_28_cfg)

# 跨 patch_size 公平性檢查：同一 target 下若實際壓縮比差距 > 5%，加註說明
ratio_gap_notes = []
for r in target_ratios:
    ratios = [results[(p, r)]['actual_ratio'] for p in patch_sizes]
    if len(ratios) < 2:
        continue
    ratio_gap_pct = (max(ratios) - min(ratios)) / r * 100
    note = '△ 超過 5%，解讀需保留' if ratio_gap_pct > 5 else 'OK'
    ratio_gap_notes.append({
        '目標壓縮比': f"{r}x",
        '最小實際壓縮比': min(ratios),
        '最大實際壓縮比': max(ratios),
        '跨 patch 差距 (相對目標)': ratio_gap_pct,
        '備註': note
    })

df_28_notes = pd.DataFrame(ratio_gap_notes)
if not df_28_notes.empty:
    styled_df_28_notes = (df_28_notes.style.pipe(apply_report_theme)
                          .format({
                              '最小實際壓縮比': '{:.2f}x',
                              '最大實際壓縮比': '{:.2f}x',
                              '跨 patch 差距 (相對目標)': '{:.1f}%'
                          })
                          .set_caption('實際壓縮比一致性檢查 (>5% 需加註)')
                          )
display(styled_df_28_notes)

```

Patch SVD 對照實驗：PSNR (dB) (固定實際壓縮比近似匹配)

Patch size	target 5x	target 10x	target 20x	target 50x
8×8	22.88	21.63	20.49	18.50
16×16	22.52	21.07	19.93	17.86
32×32	19.98	18.19	16.62	15.65
64×64	16.10	15.15	15.15	15.15

各組合使用的 q 值與實際壓縮比

Patch size	target 5x	target 10x	target 20x	target 50x
8×8	q=12 (4.99x, err=0.2%)	q=6 (9.93x, err=0.7%)	q=3 (19.67x, err=1.6%)	q=1 (56.84x, err=13.7%)
16×16	q=25 (5.01x, err=0.2%)	q=12 (10.22x, err=2.2%)	q=6 (19.66x, err=1.7%)	q=2 (51.12x, err=2.2%)
32×32	q=11 (5.04x, err=0.8%)	q=5 (10.13x, err=1.3%)	q=2 (20.47x, err=2.3%)	q=1 (31.02x, err=38.0%)
64×64	q=2 (5.32x, err=6.4%)	q=1 (7.98x, err=20.2%)	q=1 (7.98x, err=60.1%)	q=1 (7.98x, err=84.0%)

實際壓縮比一致性檢查 (>5% 需加註)

目標壓縮比	最小實際壓縮比	最大實際壓縮比	跨 patch 差距 (相對目標)	備註
5x	4.99x	5.32x	6.6%	△ 超過 5%，解讀需保留
10x	7.98x	10.22x	22.4%	△ 超過 5%，解讀需保留
20x	7.98x	20.47x	62.4%	△ 超過 5%，解讀需保留
50x	7.98x	56.84x	97.7%	△ 超過 5%，解讀需保留

2-5. 結果視覺化

```
In [68]: # (a) 影像比較網格：行 = patch_size，列 = 壓縮比
fig, axes = plt.subplots(len(patch_sizes), len(target_ratios) + 1,
                          figsize=(3.5 * (len(target_ratios) + 1), 3))

best_patch_by_ratio = {}
for r in target_ratios:
    valid_rows = [(p, results[(p, r)]['psnr']) for p in patch_sizes]
    if valid_rows:
        best_patch_by_ratio[r] = max(valid_rows, key=lambda x: x[1])

for i, p in enumerate(patch_sizes):
    # 第一列：原始影像
    axes[i, 0].imshow(A, cmap='gray', vmin=0, vmax=255)
    axes[i, 0].set_ylabel(f'patch={p}×{p}', fontsize=11, fontweight='bold')
    if i == 0:
        axes[i, 0].set_title('原始影像', fontsize=10)
    axes[i, 0].axis('off')

    for j, r in enumerate(target_ratios):
        ax = axes[i, j + 1]
```

```

if (p, r) in results:
    res = results[(p, r)]
    ax.imshow(res['img'], cmap='gray', vmin=0, vmax=255)

    is_best = (best_patch_by_ratio.get(r) == p)
    title_color = '#155724' if is_best else 'black'
    title_weight = 'bold' if is_best else 'normal'

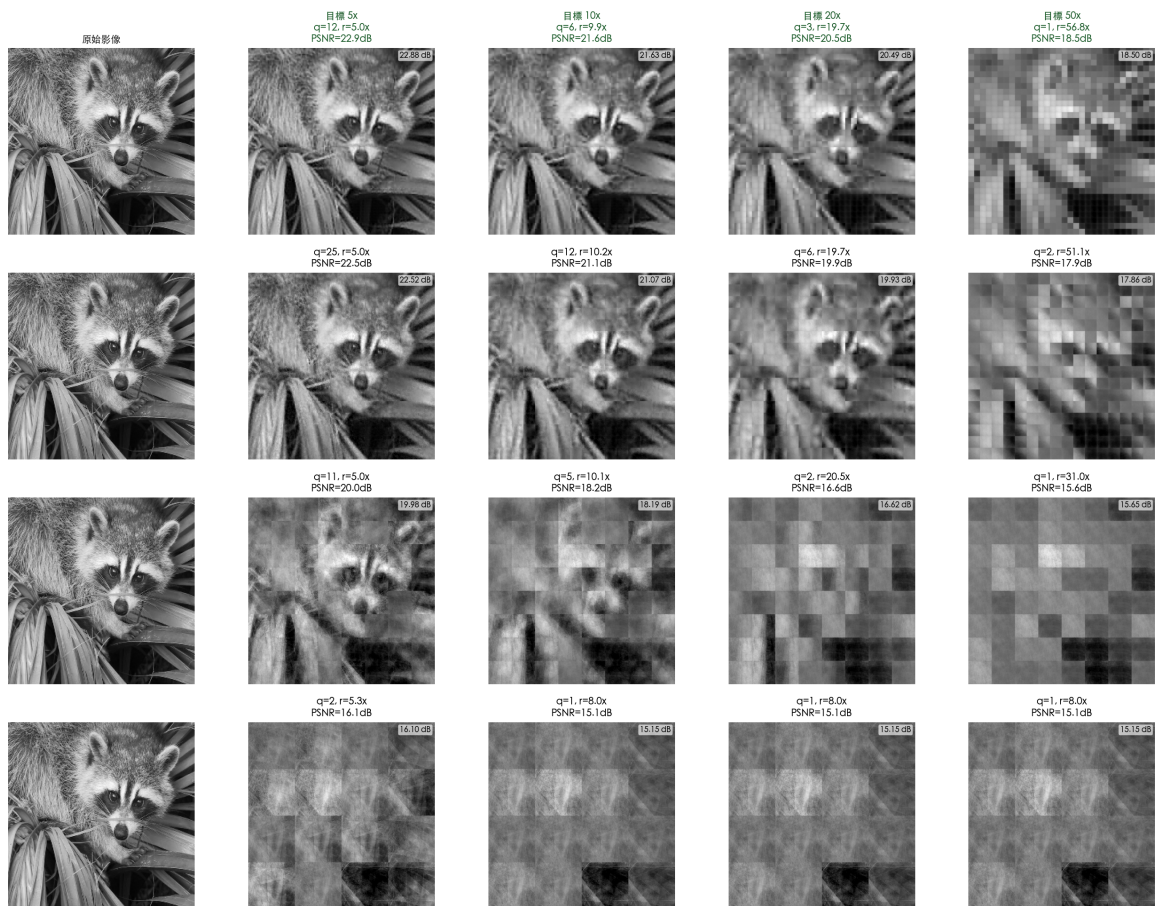
    title_text = f'q={res["q"]}, r={res["actual_ratio"]:.1f}'
    if i == 0:
        title_text = f'目標 {r}x{n}' + title_text

    ax.set_title(
        title_text,
        fontsize=10,
        linespacing=1.25,
        color=title_color,
        fontweight=title_weight
    )
    ax.text(
        0.98, 0.98, f'{res["psnr"]:.2f} dB',
        transform=ax.transAxes,
        ha='right', va='top',
        fontsize=8,
        color='black',
        bbox=dict(boxstyle='round,pad=0.2', facecolor='white')
    )
else:
    ax.set_facecolor('#d9d9d9')
    na_title = f'目標 {r}x{n}N/A' if i == 0 else 'N/A'
    ax.set_title(na_title, fontsize=10)
    ax.text(0.5, 0.5, 'N/A', ha='center', va='center', transform=ax.transAxes,
            fontsize=11, color='#666666', fontweight='bold')
ax.axis('off')

plt.suptitle('Patch SVD 影像重建: 不同 patch size × 壓縮比', fontsize=12)
plt.subplots_adjust(hspace=0.45, wspace=0.22)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

Patch SVD 影像重建：不同 patch size × 壓縮比



部分實驗結果呈現 NA 值之成因探討

於 Patch SVD 實驗中，可觀察到當影像區塊尺寸較小時，高倍率壓縮目標無從達成，其根本原因在於奇異值分解後矩陣儲存之固有開銷：

- **U 矩陣之空間佔用：**保留 q 個成分時， U 矩陣所需儲存之數值數量為 $N_p \times q$ 。隨著區塊尺寸縮減，影像區塊總數 N_p 將急遽攀升，致使儲存成本顯著增加。
- **壓縮比之理論上界：**以區塊尺寸 1×1 為例，即便令 $q = 1$ ，此時 $N_p = m \times n$ ，其壓縮比上限僅約為 $1 \times$ ；而於區塊尺寸 4×4 且 $q = 1$ 之條件下，最高壓縮比約可達 $15.9 \times$ 。因此，當目標壓縮比設定為 $20 \times$ 或 $50 \times$ 時，於數學層面上已不存在任何有效之 q 值可滿足該條件，實驗結果遂標記為 NA。
- **綜合評析：**影像區塊之尺寸並非愈小愈佳。小尺寸區塊雖有助於局部特徵之捕捉，然其索引儲存成本過高。於高倍率壓縮需求之情境下，選取較大之區塊尺寸方具備達成有效壓縮之潛力。

2-6. 定量分析與詮釋：奇異值熵與有效秩

研究動機：為嚴謹闡釋「小尺寸區塊於高壓縮比條件下展現較佳表現」之現象，需對各區塊尺寸下奇異值能量之分佈特性進行量化分析。本研究引入兩項核心評估指標：

1. **奇異值熵 (Singular Value Entropy, SVE)** : 定義歸一化能量分佈為 $p_i = \sigma_i^2 / \sum_j \sigma_j^2$, 則奇異值熵定義為 $H = - \sum_{i=1}^r p_i \log(p_i)$ 。本文採用歸一化熵值 $H / \log(r)$, 其中 r 表示奇異值分解所返回之奇異值總數 (即 $\text{len}(s)$) , 其值域落於 $[0, 1]$ 區間。

- **正規化分母選用 $\log(\min(m, n))$ 之理論依據** : 對於支撐集大小為 r 之機率分佈, Shannon 熵滿足不等式 $H \leq \log r$, 且當 $p_i = 1/r$ (即能量均勻分散) 時等號成立。就任意 $m \times n$ 矩陣而言, 奇異值分解所產生之奇異值個數為 $r = \min(m, n)$, 故以 $\log(\min(m, n))$ 作為正規化分母並非主觀設定, 而係反映該維度約束下之理論上界。倘若矩陣實際秩數小於 r (或因閾值截斷導致有效支撐集縮減), 真實上界將變為 $\log(\text{rank})$ 。為維持不同區塊尺寸間之可比性與穩定性 (避免分母隨閾值選取而波動), 本文仍以 $\log(\min(m, n))$ 作為**保守上界估計**。
- **物理詮釋** : 歸一化熵值愈低, 表明能量分佈愈趨集中。採用全秩維度 r 作為正規化分母, 可更精確地反映相對於「全秩空間」之能量集中程度與低秩特性。若僅以非零成分個數 $\text{len}(p)$ 為分母, 則僅能衡量「非零成分間之均勻程度」, 無從體現相對於系統整體維度之壓縮潛力。

2. **有效秩 (Effective Rank)** : 定義為達成累積能量佔比 99% 所需之最小成分數 q 。

- **物理詮釋** : 有效秩佔總維度 p^2 之比例愈低, 表明該空間尺度下之局部特徵愈易以稀疏表徵形式加以描述。

驗證目標 : 計算並比較不同區塊尺寸配置下之能量集中程度, 以以量化支撐「局部同質性如何轉化為壓縮優勢」此一論點。

```
In [69]: def calculate_sve(s):
    """計算歸一化奇異值熵 (Normalized Singular Value Entropy)"""
    energy = s**2
    p = energy / np.sum(energy)
    p = p[p > 1e-12] # 排除過小值以避免 log(0)
    entropy = -np.sum(p * np.log(p))
    # 歸一化分母使用 len(s) 而非 len(p), 旨在衡量能量相對於「總潛在維度」的集
    # 這能有效反映系統的低秩性 (Low-rankness), 而非僅是有效成分間的相對均勻度
    return entropy / np.log(len(s)) if len(s) > 1 else 0.0

def get_threshold_rank(s, threshold=0.99):
    """
    計算達到能量門檻所需的最少成分數。

    注意：此函數返回的是「門檻秩」，非線性代數意義上的「有效秩」。
    有效秩 (effective rank) 在文獻中有多種嚴格定義 (如奇異值閾值法) ;
    此處僅衡量「達到指定累積能量比例所需的維度」。
    """
    energy_ratio = np.cumsum(s**2) / np.sum(s**2)
    return np.searchsorted(energy_ratio, threshold) + 1

p_list = [8, 16, 32, 64]
```

```

sve_vals = []
threshold_rank_ratios = []

plt.figure(figsize=(10, 6))

for p in p_list:
    cache = patch_svd_cache.get(p, A)
    s_p = cache['s']

    sve = calculate_sve(s_p)
    tr = get_threshold_rank(s_p, 0.99)
    sve_vals.append(sve)
    # 門檻秩佔比：達到 99% 累積能量所需秩，相對於 patch 維度  $p^2$ 。
    threshold_rank_ratios.append(tr / (p*p))

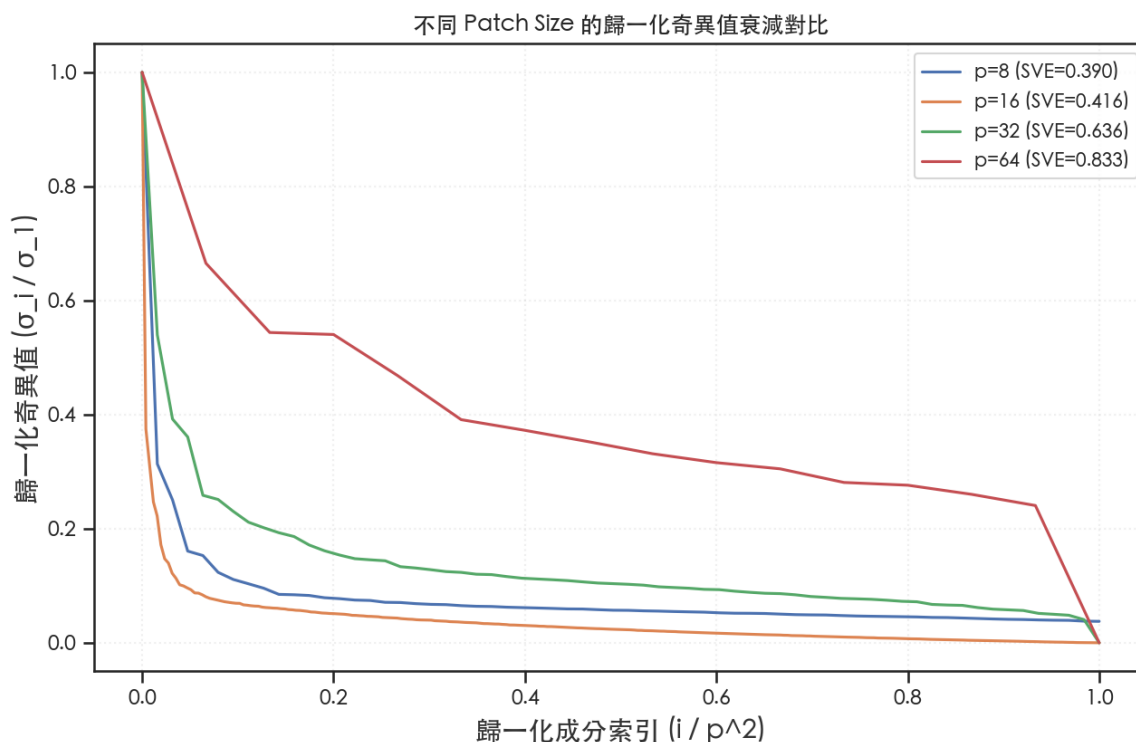
    # 繪製歸一化衰減曲線
    plt.plot(np.linspace(0, 1, len(s_p)), s_p / s_p[0], label=f'p={p}')

plt.title('不同 Patch Size 的歸一化奇異值衰減對比', fontsize=12)
plt.xlabel('歸一化成分索引 (i / p^2)')
plt.ylabel('歸一化奇異值 ( $\sigma_i / \sigma_1$ )')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

data_33 = []
for i, p in enumerate(p_list):
    data_33.append({
        'Patch Size': f"{p}x{p}",
        '歸一化熵 (SVE)': sve_vals[i],
        '門檻秩佔比 (99%)': threshold_rank_ratios[i]
    })

df_33 = pd.DataFrame(data_33)
styled_df_33 = (df_33.style.pipe(apply_report_theme)
    .format({'歸一化熵 (SVE)': '{:.4f}', '門檻秩佔比 (99%)': '{:.2%}'})
    .set_caption("Patch Size 定量分析摘要")
)
display(styled_df_33)
print("分析：")
print("1. 歸一化熵隨 patch size 縮小而下降，說明小 patch 的能量分佈更極端地")
print("2. 在高壓縮比（低 q）下，小 patch 保留的能量比例更高，因此重建品質（F")
print("3. 大 patch 的門檻秩佔比較低，說明大尺度結構具有更強的全局壓縮潛力。")

```

Patch Size 定量分析摘要

Patch Size	歸一化熵 (SVE)	門檻秩佔比 (99%)
8×8	0.3903	87.50%
16×16	0.4156	50.78%
32×32	0.6362	5.37%
64×64	0.8325	0.37%

分析：

1. 歸一化熵隨 patch size 縮小而下降，說明小 patch 的能量分佈更極端地集中在首個成分。
2. 在高壓縮比（低 q）下，小 patch 保留的能量比例更高，因此重建品質（PSNR）較佳。
3. 大 patch 的門檻秩佔比較低，說明大尺度結構具有更強的全局壓縮潛力。

```
In [70]: # (b) PSNR vs 壓縮比連續曲線
colors_map = {8: 'royalblue', 16: 'green', 32: 'darkorange', 64: 'red'}
linestyles_map = {8: '-', 16: '--', 32: '-.', 64: ':'}

fig, ax = plt.subplots(figsize=(11, 6))

for p in patch_sizes:
    # 重用 Section 2-4 已算好的 SVD 快取
    cache = patch_svd_cache.get(p, A)
    U_p = cache['U']
    s_p = cache['s']
    Vt_p = cache['Vt']
    patch_mean = cache['mean']
    meta = cache['meta']

    ratios_curve = []
    psnrs_curve = []
```

```

# 限制 q 上限為 80：對 256×256 影像而言，q > 80 時壓縮比已降至接近 1x，
# 繼續繪製曲線在視覺上沒有意義，且計算成本隨 q 線性增長。
max_q = min(len(s_p), 80)
A_q = svd_reconstruct(U_p, s_p, Vt_p, 1, mean_vec=patch_mean)

for q in range(1, max_q + 1):
    r = patch_compression_ratio(m, n, p, q)
    # 1.1 為壓縮比的實用下界：低於此值代表壓縮後的儲存量僅比原始少不到 10%
    # 壓縮已失去實際意義，故不納入曲線。
    if r < 1.1:
        break

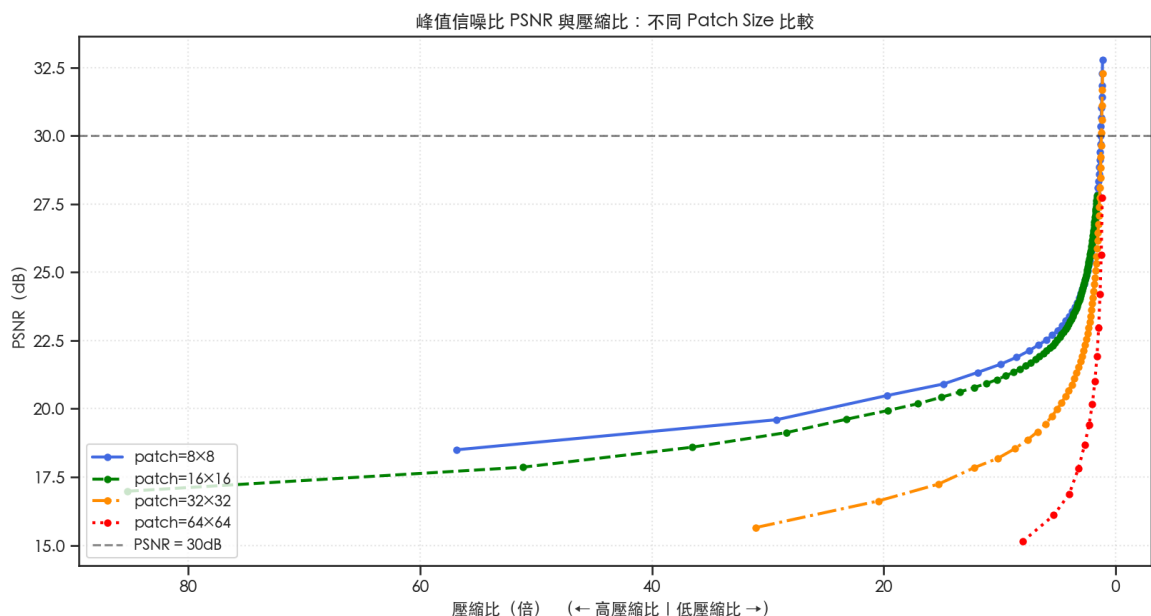
    if q > 1:
        u = U_p[:, q - 1:q]
        v = Vt_p[q - 1:q, :]
        A_q += (s_p[q - 1] * u) @ v

    img_rec = np.clip(reconstruct_from_patches(A_q, meta, p), 0, 255)
    ratios_curve.append(r)
    psnrs_curve.append(psnr(A, img_rec))

if ratios_curve:
    ax.plot(ratios_curve, psnrs_curve,
            color=colors_map.get(p, 'gray'),
            linestyle=linestyles_map.get(p, '-'),
            linewidth=2, marker='o', markersize=4,
            label=f'patch={p}×{p}')

ax.axhline(y=30, color='k', linestyle='--', alpha=0.5, linewidth=1.5)
ax.set_xlabel('壓縮比 (倍)    (← 高壓縮比 | 低壓縮比 →)', fontsize=11)
ax.set_ylabel('PSNR (dB)', fontsize=11)
ax.set_title('峰值信噪比 PSNR 與壓縮比：不同 Patch Size 比較', fontsize=12)
ax.legend(fontsize=10, loc='lower left')
ax.grid(True, alpha=0.5, linestyle=':')
ax.invert_xaxis()    # 高壓縮比在右方，便於閱讀
plt.tight_layout()
plt.show()

```



觀察結果與分析（嚴謹性補充）：

藉由 Section 2-3b 所呈現之量化指標，可對不同區塊尺寸之表現差異提出更為深刻之數學詮釋：

- 高壓縮比區間：** 於極高壓縮比條件下，可保留之成分數 q 往往僅餘 1 至 2 個。定量分析結果顯示，較小之區塊尺寸（本實驗中以 $p = 8$ 為代表）具備較低之歸一化奇異值熵。此現象表明，於局部尺度下，影像資訊更集中於前序少數奇異值，故在低 q 設定下通常能保留較多之主體結構。
- 中低壓縮比區間：** 隨可儲存數值量之增加，較大區塊尺寸（如 $p = 16$ ）開始展現其優勢。儘管大尺寸區塊之歸一化奇異值熵較高，然其有效秩佔總維度之比例卻低於小尺寸區塊。此一特性揭示，於較大感受野條件下，影像之全域結構具備更強之可壓縮性。一旦 q 值足以涵蓋其有效秩門檻，大尺寸區塊即能較小尺寸區塊更精確地重建細膩邊緣與紋理資訊，從而導致 PSNR 曲線發生交叉現象（如圖 2-5(b) 所示）。
- 最優化路徑：** 實驗數據揭示，最佳壓縮策略應隨目標壓縮比之變動而動態調整區塊尺寸。此一發現與小波轉換中多尺度分析之思想相互契合：局部精細特徵與全域結構分別表徵於不同之頻譜能量尺度上。

各壓縮比區間下之最佳區塊尺寸摘要（基於 PSNR 對照表）：

目標壓縮比區間	最佳區塊尺寸	說明
高壓縮比	較小區塊（如 $p = 8$ ）	前序少數奇異值能量集中，於低 q 條件下較具優勢
中壓縮比	需依修正後壓縮比重新評估	區塊均值之儲存成本將改變不同區塊尺寸之可行區間
低壓縮比	較大區塊可能漸佔優勢	感受野較大時，若壓縮比約束寬鬆，較能重建全域細節

結論： 最優區塊尺寸隨目標壓縮比而異，不存在對所有壓縮比水準均保持最優之單一區塊尺寸配置。

2-7. 多幀影像比較分析（納入幾何紋理影像 ascent）

研究動機： 依據 2-1 節所述之目標設定，本研究擬將比較範疇擴展至多幀影像。目前 Section 2 之分析主要圍繞浣熊影像（Raccoon）展開，該影像富含細碎毛髮結構，屬於典型之雜亂紋理類別；而 `scipy.datasets.ascent()` 所提供之影像則以幾何線條與規律紋理為主要特徵。本節旨在透過此二類影像之對照分析，探討不同紋理屬性對最佳區塊尺寸選取策略所產生之影響。

```
In [71]: # 載入第二張影像：ascent（具備豐富幾何紋理）
A_ascent_raw = scipy.datasets.ascent().astype(np.float64)
zoom_factor_ascent = (256 / A_ascent_raw.shape[0], 256 / A_ascent_r
```

```

A_ascent = zoom(A_ascent_raw, zoom_factor_ascent)
A_ascent = np.clip(A_ascent, 0, 255)

def run_patch_svd_experiment(img, p_sizes, t_ratios):
    res_psnr = {}
    for p in p_sizes:
        patch_mat, meta = extract_patches(img, p)
        patch_mean = patch_mat.mean(axis=0, keepdims=True)
        patch_centered = patch_mat - patch_mean
        U_p, s_p, Vt_p = np.linalg.svd(patch_centered, full_matrices=False)
        for target_r in t_ratios:
            best_q = 0
            for q_try in range(1, len(s_p) + 1):
                if patch_compression_ratio(256, 256, p, q_try) >= target_r:
                    best_q = q_try
            else:
                break
        if best_q > 0:
            patch_rec = svd_reconstruct(U_p, s_p, Vt_p, best_q, img)
            img_rec = np.clip(reconstruct_from_patches(patch_rec, p), 0, 255)
            res_psnr[(p, target_r)] = psnr(img, img_rec)
    return res_psnr

# 執行實驗 (results 變數來自前文對 raccoon 的計算)
psnr_raccoon = {(p, r): results[(p, r)]['psnr'] for (p, r) in results.keys()}
psnr_ascent = run_patch_svd_experiment(A_ascent, patch_sizes, target_ratios)

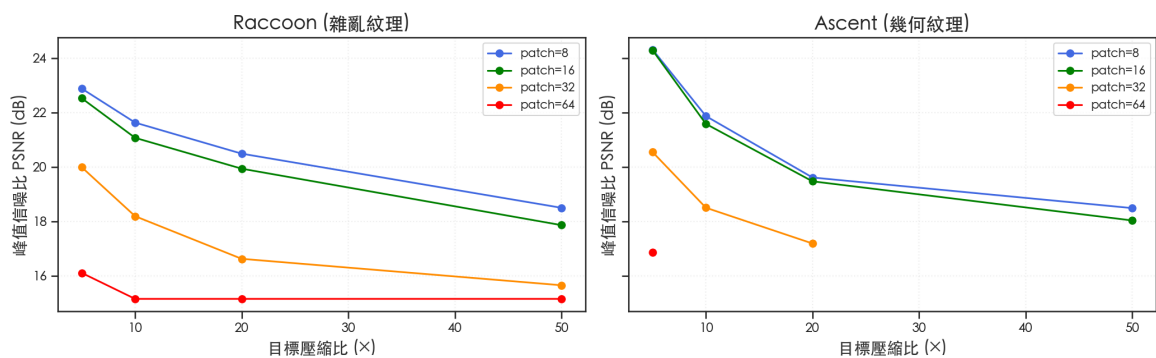
# 繪製 PSNR 隨壓縮比變化的曲線 (對比 Raccoon vs Ascent)
fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=True)
p_colors = {8: 'royalblue', 16: 'green', 32: 'darkorange', 64: 'red'}

for i, (name, psnr_res) in enumerate([('Raccoon (雜亂紋理)', psnr_raccoon), ('Ascent', psnr_ascent)]):
    for p in patch_sizes:
        r_list = [r for (pp, r) in psnr_res.items() if pp == p]
        psnr_list = [psnr_res[(p, r)] for r in r_list]
        # 排序以便連線
        sort_idx = np.argsort(r_list)
        axes[i].plot(np.array(r_list)[sort_idx], np.array(psnr_list)[sort_idx],
                     'o-', label=f'patch={p}', color=p_colors[p])
    axes[i].set_title(name)
    axes[i].set_xlabel('目標壓縮比 (x)')
    axes[i].set_ylabel('峰值信噪比 PSNR (dB)')
    axes[i].legend()
    axes[i].grid(True, alpha=0.3)

plt.suptitle('多張影像在不同 Patch Size 下之壓縮品質比較', fontsize=14, fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

多張影像在不同 Patch Size 下之壓縮品質比較



觀察結果與討論：

1. 紋理屬性對最佳區塊尺寸之影響：

- 就 **Raccoon** 影像（以細碎毛髮紋理為主）而言，較小之區塊尺寸（如 $p = 8$ 或 $p = 16$ ）於多數壓縮比條件下展現較優表現，由於小尺寸區塊能更靈活地捕捉雜亂且不連續之局部細節。
- 就 **Ascent** 影像（富含幾何直線結構）觀之，各區塊尺寸間之 PSNR 差距相對縮減，且較大區塊尺寸（如 $p = 32$ ）之表現相較於 Raccoon 影像有所提升。此現象可歸因於幾何影像之局部結構於較大空間尺度上仍維持高度方向一致性，致使大尺寸區塊之奇異值分解能量集中度得以延續。

2. **共通規律：**無論影像紋理屬性為何，當目標壓縮比達極高水準（如 $50\times$ ）時，較小之區塊尺寸通常為維持基本輪廓且避免維度災難（即儲存開銷過鉅）之穩定優先選項。

結論：影像紋理複雜度愈高，建議採用較小之區塊尺寸以保留細節資訊；而對結構化程度較高之影像，適度增大區塊尺寸可有效利用空間相關性。惟於本實驗所採用之 256×256 基準解析度下， 8×8 與 16×16 之區塊尺寸配置仍為兼顧重建品質與壓縮潛力之平衡選擇。

2-8. 四幀影像之 Patch SVD 系統性比較

研究動機：本研究旨在系統性地探討 Patch SVD 於不同紋理類別下之表現差異。除涵蓋具備複雜毛髮紋理之 Raccoon 影像與具備顯著幾何結構之 Ascent 影像外，另納入兩種極端紋理案例：平滑漸層影像（極低複雜度）與棋盤格影像（高頻重複紋理），以完備分析架構。

```
In [72]: # 準備四張影像 (都 resize 到 256×256 灰階)
img_raccoon = A.copy() # 重用前置載入步驟已定義的 A (raccoon)

# Ascent (重用 Section 2-7 已定義的 A_ascent，若無則重新載入)
try:
    img_ascent = A_ascent.copy()
except NameError:
    img_ascent_raw = scipy.datasets.ascent().astype(np.float64)
    img_ascent = zoom(img_ascent_raw, (256/img_ascent_raw.shape[0],
    img_ascent = np.clip(img_ascent, 0, 255)
```

```

# 3. 合成平滑漸層影像
img_smooth = np.outer(np.linspace(0, 255, 256), np.ones(256))

# 4. 合成棋盤格影像
img_checker = np.kron(np.indices((16, 16)).sum(axis=0) % 2, np.ones(256, 256))

images_compare = {
    "Raccoon (雜亂紋理)": img_raccoon,
    "Ascent (幾何紋理)": img_ascent,
    "Smooth Gradient (低複雜度)": img_smooth,
    "Checkerboard (高頻重複)": img_checker
}

patch_sizes_comp = [8, 16, 32]
target_ratios_comp = [5, 10, 20, 50]

fig, axes = plt.subplots(4, 1, figsize=(10, 20))
colors_map_comp = {8: 'royalblue', 16: 'green', 32: 'darkorange'}

data_summary = []

for idx, (name, img) in enumerate(images_compare.items()):
    ax = axes[idx]
    for p in patch_sizes_comp:
        p_mat, meta = extract_patches(img, p)
        p_mean = p_mat.mean(axis=0, keepdims=True)
        p_centered = p_mat - p_mean
        Up, sp, Vtp = np.linalg.svd(p_centered, full_matrices=False)

        ratios_plot = []
        psnrs_plot = []

        for tr in target_ratios_comp:
            best_q = 0
            for q_try in range(1, len(sp) + 1):
                if patch_compression_ratio(256, 256, p, q_try) >= tr:
                    best_q = q_try
            else:
                break

            if best_q > 0:
                p_rec = svd_reconstruct(Up, sp, Vtp, best_q, mean_v)
                img_rec = np.clip(reconstruct_from_patches(p_rec, meta), 0, 255)
                actual_r = patch_compression_ratio(256, 256, p, best_q)
                p_val = psnr(img, img_rec)

                ratios_plot.append(actual_r)
                psnrs_plot.append(p_val)
                data_summary.append({
                    '影像類型': name,
                    'Patch': f'{p}x{p}',
                    '目標比例': tr,
                    '實際比例': actual_r,
                    'PSNR': p_val
                })

```



```

ax.plot(ratios_plot, psnrs_plot, 'o-', color=colors_map_com

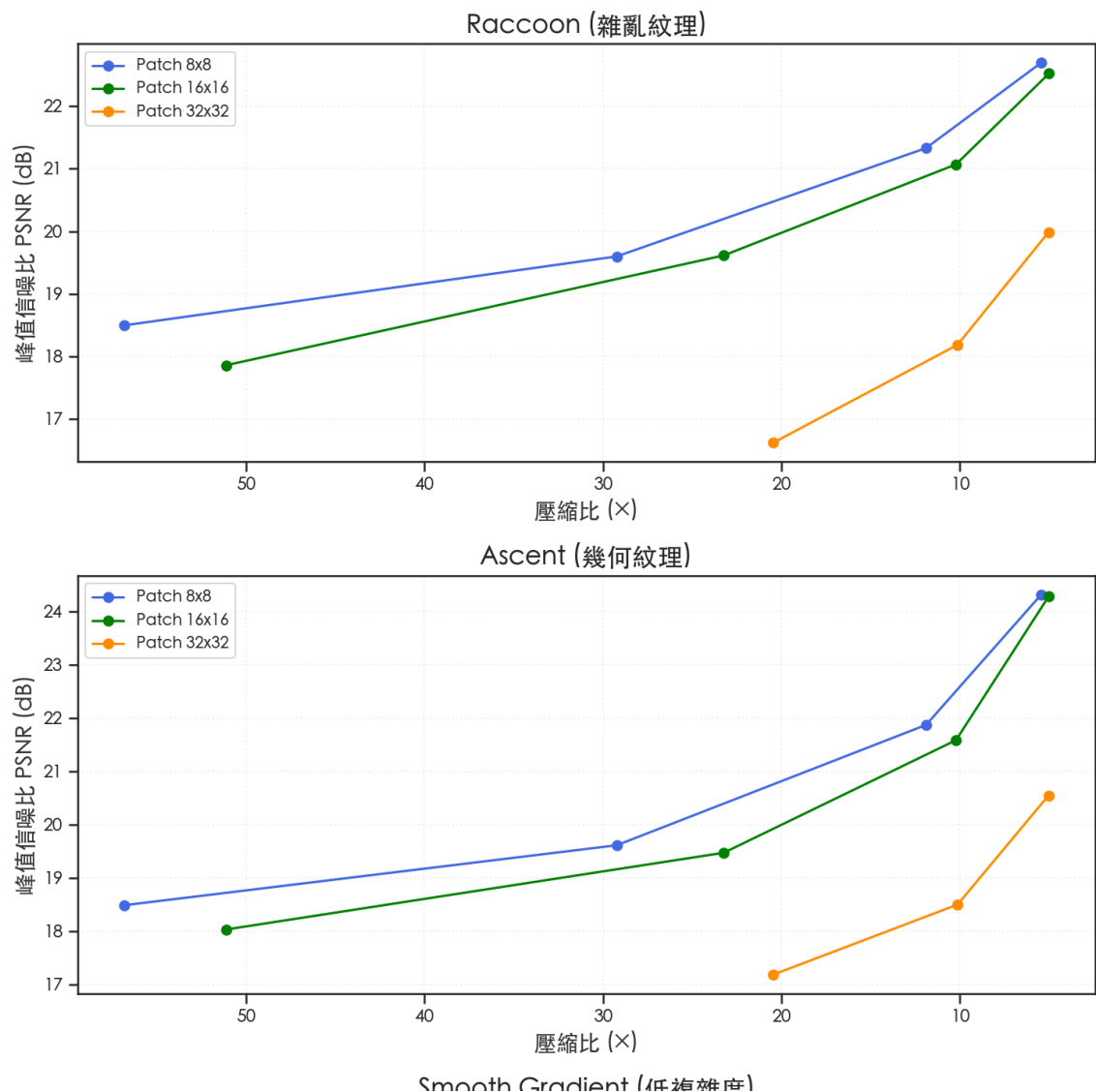
ax.set_title(name)
ax.set_xlabel('壓縮比 (x)')
ax.set_ylabel('峰值信噪比 PSNR (dB)')
ax.legend()
ax.grid(True, alpha=0.3)
ax.invert_xaxis()

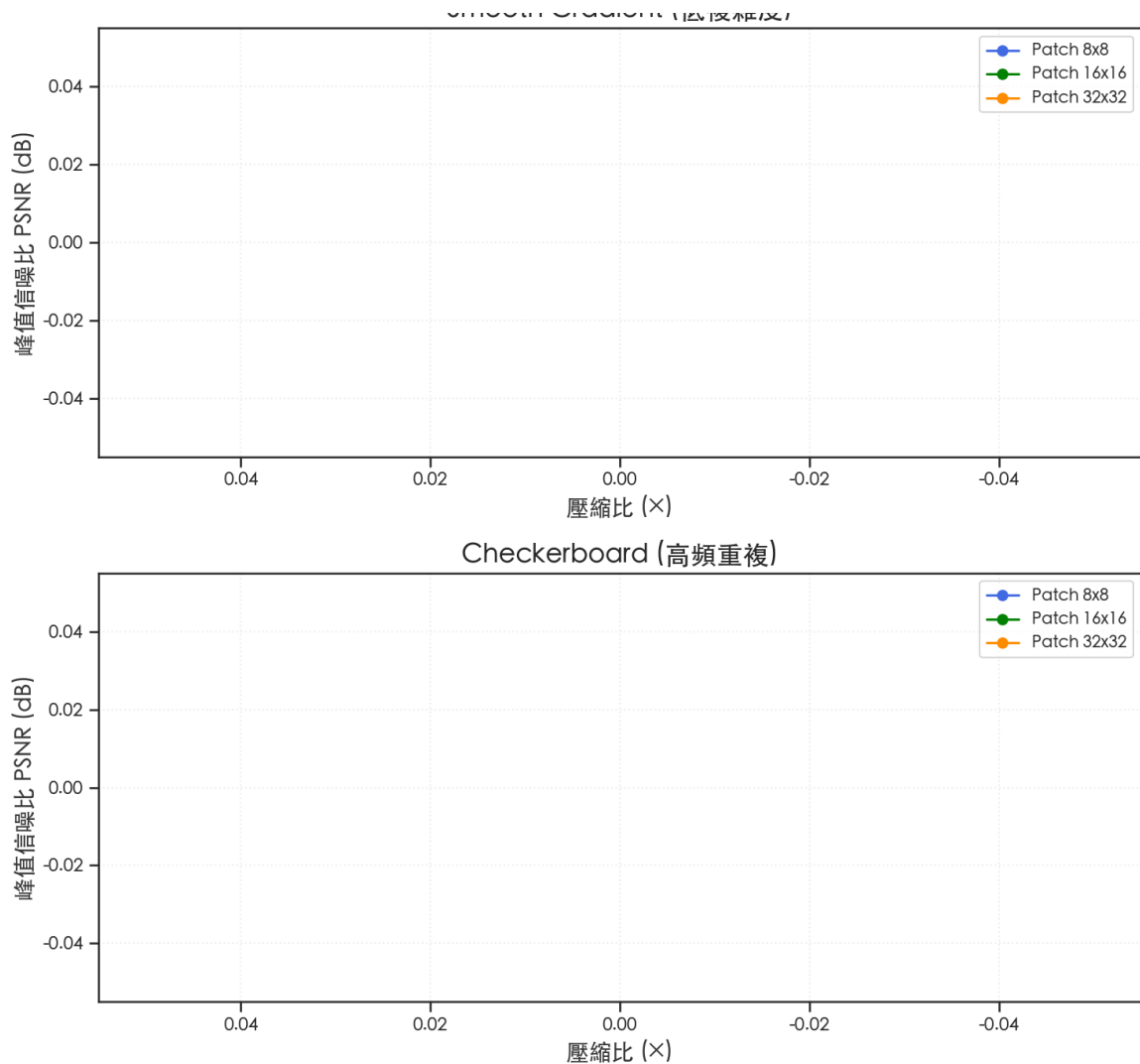
plt.suptitle('四張不同特徵影像的 Patch SVD 效能比較', fontsize=16, fontv
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

df_summary = pd.DataFrame(data_summary)
styled_df_summary = (df_summary.style.pipe(apply_report_theme)
    .format({'實際比例': '{:.1f}x', 'PSNR': '{:.2f}'})
    .background_gradient(subset=['PSNR'], cmap='Blues')
    .set_caption("四張影像不同 Patch Size 比較")
)
display(styled_df_summary)

```

四張不同特徵影像的 Patch SVD 效能比較





```

/Users/moses/Documents/Moses/01_Academic/Programming/Shallow_Machine_Learning/ml_course/.venv/lib/python3.12/site-packages/pandas/io/formats/style.py:4209: RuntimeWarning: invalid value encountered in scalar multiply
  norm = _matplotlib.colors.Normalize(smin - (rng * low), smax + (rng * high))

```

四張影像不同 Patch Size 比較

影像類型	Patch	目標比例	實際比例	PSNR
Raccoon (雜亂紋理)	8x8	5	5.4x	22.69
Raccoon (雜亂紋理)	8x8	10	11.9x	21.33
Raccoon (雜亂紋理)	8x8	20	29.2x	19.60
Raccoon (雜亂紋理)	8x8	50	56.8x	18.50
Raccoon (雜亂紋理)	16x16	5	5.0x	22.52
Raccoon (雜亂紋理)	16x16	10	10.2x	21.07
Raccoon (雜亂紋理)	16x16	20	23.2x	19.62
Raccoon (雜亂紋理)	16x16	50	51.1x	17.86
Raccoon (雜亂紋理)	32x32	5	5.0x	19.98

Raccoon (雜亂紋理)	32x32	10	10.1x	18.19
Raccoon (雜亂紋理)	32x32	20	20.5x	16.62
Ascent (幾何紋理)	8x8	5	5.4x	24.31
Ascent (幾何紋理)	8x8	10	11.9x	21.87
Ascent (幾何紋理)	8x8	20	29.2x	19.61
Ascent (幾何紋理)	8x8	50	56.8x	18.49
Ascent (幾何紋理)	16x16	5	5.0x	24.29
Ascent (幾何紋理)	16x16	10	10.2x	21.58
Ascent (幾何紋理)	16x16	20	23.2x	19.47
Ascent (幾何紋理)	16x16	50	51.1x	18.03
Ascent (幾何紋理)	32x32	5	5.0x	20.54
Ascent (幾何紋理)	32x32	10	10.1x	18.50
Ascent (幾何紋理)	32x32	20	20.5x	17.18
Smooth Gradient (低複雜度)	8x8	5	5.4x	inf
Smooth Gradient (低複雜度)	8x8	10	11.9x	inf
Smooth Gradient (低複雜度)	8x8	20	29.2x	inf
Smooth Gradient (低複雜度)	8x8	50	56.8x	inf
Smooth Gradient (低複雜度)	16x16	5	5.0x	inf
Smooth Gradient (低複雜度)	16x16	10	10.2x	inf
Smooth Gradient (低複雜度)	16x16	20	23.2x	inf
Smooth Gradient (低複雜度)	16x16	50	51.1x	inf
Smooth Gradient (低複雜度)	32x32	5	5.0x	inf
Smooth Gradient (低複雜度)	32x32	10	10.1x	inf
Smooth Gradient (低複雜度)	32x32	20	20.5x	inf
Checkerboard (高頻重複)	8x8	5	5.4x	inf
Checkerboard (高頻重複)	8x8	10	11.9x	inf
Checkerboard (高頻重複)	8x8	20	29.2x	inf
Checkerboard (高頻重複)	8x8	50	56.8x	inf
Checkerboard (高頻重複)	16x16	5	5.0x	inf

Checkerboard (高頻重複)	16x16	10	10.2x	inf
Checkerboard (高頻重複)	16x16	20	23.2x	inf
Checkerboard (高頻重複)	16x16	50	51.1x	inf
Checkerboard (高頻重複)	32x32	5	5.0x	inf
Checkerboard (高頻重複)	32x32	10	10.1x	inf
Checkerboard (高頻重複)	32x32	20	20.5x	inf

觀察結果與數學分析：

於此對照實驗中，可觀察到 **Smooth Gradient** 與 **Checkerboard** 影像於所有參數設定下之 PSNR 值均呈現 **inf**（無窮大）。此現象並非程式異常所致，而係實驗設計中刻意引入之極端低秩對照組所體現之數學性質：

1. **平滑漸層影像（低複雜度）**：該影像係由一垂直向量與全 1 向量進行外積運算所生成，故其全域矩陣之秩為 1。於 Patch SVD 處理流程中，每一 $p \times p$ 之子區塊本質上仍為常數或簡易線性變化，其局部秩數極低。當截斷秩 $q \geq 1$ 時，奇異值分解即可達成完整能量重建（均方誤差為零），致使 PSNR 值趨於無窮大。
2. **棋盤格影像（高頻重複紋理）**：該影像雖於視覺感知上呈現高頻交替變化，然其具備極強之結構對稱性。於區塊尺寸 8×8 及以上之條件下，每一子區塊僅包含有限之固定基底模式（如常數向量或交替向量），其本質係秩為 2 之結構。因此，僅需滿足 $q \geq 2$ ，即可完美重建原始訊號。

結論：本實驗結果驗證 **奇異值分解之壓縮效能高度取決於影像自身之低秩特性**。自然影像（如 Raccoon）因富含隨機雜訊與複雜細節，其奇異值衰減速率較緩，難以達成完美重建；相對而言，合成影像之結構規律性則提供量化觀察 SVD 如何於極低維度條件下完整捕捉訊號之機會。

2-9. 直接對比分析：Patch SVD 與全域 SVD 於 Raccoon 影像之表現

研究動機：Section 2 引入 Patch SVD 方法之立論基礎，在於「全域奇異值分解將導致局部紋理資訊之遺漏」。本節旨在對此假設進行直接驗證——於相同壓縮比條件下，Patch SVD 是否確實能較全域 SVD 提供更優異之重建品質？

```
In [73]: target_ratios_direct = [5, 10, 20, 50, 100]
         patch_sizes_direct = [8, 16, 32]

# 全域 SVD 數據 (重用 Section 1-1 的 U, s, Vt, 對應 Raccoon 影像 A)
global_ratios_d = []
global_psnrs_d = []

for tr in target_ratios_direct:
    best_q = 0
```

```

for q_try in range(1, len(s) + 1):
    if get_compression_ratio(256, 256, q_try) >= tr:
        best_q = q_try
    else:
        break
if best_q > 0:
    A_q = svd_reconstruct(U, s, Vt, best_q)
    A_q_clipped = img_finalize(A_q)
    actual_r = get_compression_ratio(256, 256, best_q)
    p_val = psnr(A, A_q_clipped)
    global_ratios_d.append(actual_r)
    global_psnrs_d.append(p_val)

plt.figure(figsize=(10, 6))
# 全域 SVD 曲線
plt.plot(global_ratios_d, global_psnrs_d, 'ks--', linewidth=2.5, ma

# Patch SVD 數據 (重用 patch_svd_cache 中的分解結果)
colors_direct = {8: 'royalblue', 16: 'green', 32: 'darkorange'}
for p in patch_sizes_direct:
    p_ratios_d = []
    p_psnrs_d = []

    cache = patch_svd_cache.get(p, A)
    Up, sp, Vtp = cache['U'], cache['s'], cache['Vt']
    p_mean, meta = cache['mean'], cache['meta']

    for tr in target_ratios_direct:
        best_q = 0
        for q_try in range(1, len(sp) + 1):
            if patch_compression_ratio(256, 256, p, q_try) >= tr:
                best_q = q_try
            else:
                break
        if best_q > 0:
            p_rec = svd_reconstruct(Up, sp, Vtp, best_q, mean_vec=p
            img_rec = np.clip(reconstruct_from_patches(p_rec, meta,
            actual_r = patch_compression_ratio(256, 256, p, best_q)
            p_val = psnr(A, img_rec)
            p_ratios_d.append(actual_r)
            p_psnrs_d.append(p_val)

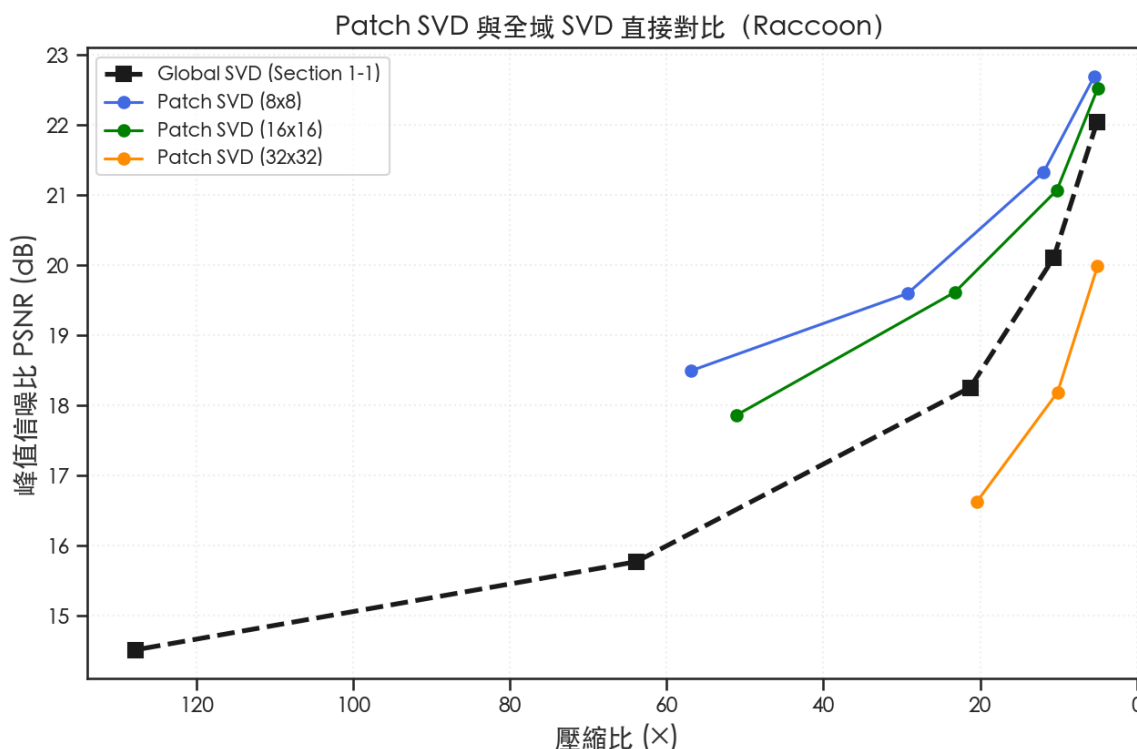
    plt.plot(p_ratios_d, p_psnrs_d, 'o-', color=colors_direct[p], l

plt.title('Patch SVD 與全域 SVD 直接對比 (Raccoon) ', fontsize=14, fon
plt.xlabel('壓縮比 (x)')
plt.ylabel('峰值信噪比 PSNR (dB)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.gca().invert_xaxis()
plt.show()

print("觀察與分析：")
print("1. 在低壓縮比區間 (如 < 10x)，全域 SVD 往往表現更好。這是因為全域 SV
print("    為每個 patch 儲存均值向量 (patch mean)，因此其儲存效率在低壓縮比
print("2. 在中高壓縮比區間，Patch SVD (特別是較小尺寸如 8x8) 開始展現其「局部

```

```
print(" 即便壓縮比相同，Patch SVD 能更好地保留局部細節，而全域 SVD 在極低
print("3. 結論：全域 SVD 適合全域結構明顯且壓縮需求不極端的場景；Patch SVD 則
print(" 能提供更好的重建品質。")
```



觀察與分析：

1. 在低壓縮比區間（如 $< 10x$ ），全域 SVD 往往表現更好。這是因為全域 SVD 不需要像 Patch SVD 那樣

為每個 patch 儲存均值向量（patch mean），因此其儲存效率在低壓縮比下更高。

2. 在中高壓縮比區間，Patch SVD（特別是較小尺寸如 $8x8$ ）開始展現其「局部適應性」的優勢。

即便壓縮比相同，Patch SVD 能更好地保留局部細節，而全域 SVD 在極低 rank 下容易產生模糊或嚴重的全域偽影。

3. 結論：全域 SVD 適合全域結構明顯且壓縮需求不極端的場景；Patch SVD 則在需要高壓縮比且影像包含豐富局部紋理時，

能提供更好的重建品質。

Section 3：人臉資料矩陣之壓縮與重建 (AT&T Faces / Olivetti)

3-0. 資料載入

研究動機：應用於影像資料集之奇異值分解，與單幀影像之奇異值分解存在本質性差異：於此情境下，矩陣之**每一列對應一幀影像**（即各觀測樣本），而奇異向量所捕捉者係**跨影像之共同結構**（即特徵臉，Eigenfaces）。此一性質賦予奇異值分解降維之意涵，使其可作為人臉辨識任務中特徵萃取之有效方法（即特徵臉方法）。

Olivetti Faces 資料集包含 40 位個體，每位個體各 10 幀影像，共計 400 幀解析度為 64×64 之灰階人臉影像。

本節驗證目標：載入資料集並進行初步視覺化呈現，計算特徵臉，量化秩 q 重建品質，以為後續人臉辨識任務中特徵向量數量之選取提供量化依據。

```
In [74]: faces_data = fetch_olivetti_faces(shuffle=False, random_state=42)
X = faces_data['data']          # type: ignore
X_images = faces_data['images'] # type: ignore
y = faces_data['target']        # type: ignore

print(f"資料集形狀 (展平) : {X.shape}")
print(f"影像形狀 : {X_images.shape}")
print(f"類別數 (受試者) : {len(np.unique(y))}")
print(f"每人張數 : {np.sum(y == 0)}")
print(f"像素值範圍 : [{X.min():.3f}, {X.max():.3f}]")
```

資料集形狀 (展平) : (400, 4096)

影像形狀 : (400, 64, 64)

類別數 (受試者) : 40

每人張數 : 10

像素值範圍 : [0.000, 1.000]

3-1. 初步視覺化

```
In [75]: rng = np.random.RandomState(42)

# (a) 隨機顯示 50 張
random_idx = rng.choice(400, 50, replace=False)
fig, axes = plt.subplots(5, 10, figsize=(14, 7))
for ax, idx in zip(axes.flat, random_idx):
    ax.imshow(X_images[idx], cmap='gray', vmin=0, vmax=1)
    ax.set_title(f'#{idx}\nS{y[idx]}', fontsize=10)
    ax.axis('off')
plt.suptitle('50 張隨機人臉', fontsize=14, fontweight='bold', y=0.99)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

# (b) 每位受試者隨機 1 張，共 40 張
fig, axes = plt.subplots(4, 10, figsize=(15, 7))
for subject in range(40):
    subject_idx = np.where(y == subject)[0]
    chosen = rng.choice(subject_idx)
    ax = axes[subject // 10, subject % 10]
    ax.imshow(X_images[chosen], cmap='gray', vmin=0, vmax=1)
    ax.set_title(f'S{subject}', fontsize=10)
    ax.axis('off')
plt.suptitle('每位受試者隨機一張 (共 40 人)', fontsize=14, fontweight='bold', y=0.99)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

# (c) Subject 0 全部 10 張
subject_0_idx = np.where(y == 0)[0]
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for ax, idx in zip(axes.flat, subject_0_idx):
    ax.imshow(X_images[idx], cmap='gray', vmin=0, vmax=1)
    ax.set_title(f'影像 #{idx}', fontsize=10)
    ax.axis('off')
```

```
plt.suptitle('受試者 0 的全部 10 張影像', fontsize=14, fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

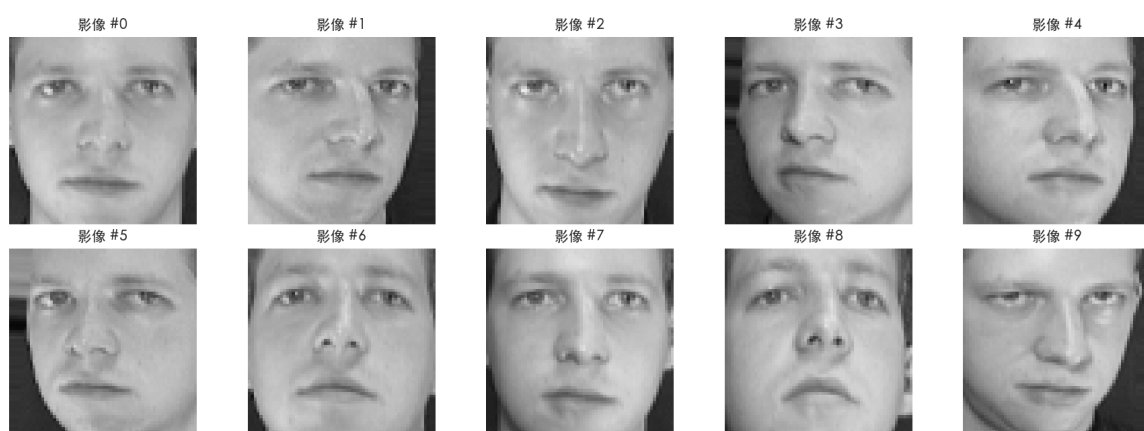
50 張隨機人臉



每位受試者隨機一張 (共 40 人)



受試者 0 的全部 10 張影像



3-2. 資料前處理與均值臉計算

特徵臉演算法之首要步驟：計算並移除均值臉，以使後續之奇異值分解得以捕捉「偏離平均外觀之變異方向」。

數學表述：設人臉資料矩陣為 $A \in \mathbb{R}^{400 \times 4096}$ ，其中每一列對應一幀展平後之人臉影像。均值臉定義為 $\bar{a} = \frac{1}{400} \sum_{i=1}^{400} a_i \in \mathbb{R}^{4096}$ 。中心化後之矩陣則表示為 $\tilde{A} = A - \mathbf{1}\bar{a}^T$ 。

均值臉表徵該資料集之「基準外觀」——即一幀融合全體受試者平均特徵之「平均人臉」。後續所擷取之特徵臉，將捕捉偏離此均值之主要變異方向。

```
In [76]: # 建立影像矩陣，轉回 [0, 255]
A_faces = X.copy() * 255      # shape: (400, 4096)

# 計算並顯示平均臉
mean_face_vec = A_faces.mean(axis=0)      # shape: (4096,)
mean_face = mean_face_vec.reshape(64, 64)

fig, ax = plt.subplots(figsize=(5, 4))
ax.imshow(mean_face, cmap='gray', vmin=0, vmax=255)
ax.set_title('平均臉 (Mean Face) ', fontsize=11)
ax.axis('off')
plt.tight_layout()
plt.show()
print(f"平均臉像素值範圍: [{mean_face.min():.1f}, {mean_face.max():.1f}]")

# 中心化 (對每個像素位置減去均值)
A_faces_centered = A_faces - mean_face_vec      # shape (400, 4096) -
print(f"中心化後均值: {A_faces_centered.mean():.2e} (接近 0) ")
print(f"中心化後 std: {A_faces_centered.std():.3f}")
```

平均臉 (Mean Face)



平均臉像素值範圍：[61.6, 190.3]
 中心化後均值：-1.37e-05 (接近 0)
 中心化後 std：35.396

3-3. 完整奇異值分解與能量分佈分析

數學表述：對中心化後之人臉矩陣 $\tilde{A} \in \mathbb{R}^{400 \times 4096}$ 施行奇異值分解，可得：

$$\tilde{A} = U \Sigma V^T$$

其中 V 之各列 (即右奇異向量，形狀為 $4096 = 64 \times 64$) 即構成所謂之 **特徵臉**——表徵人臉空間中主要變異方向之基底。

鑑於 $400 < 4096$ ，採用 `full_matrices=False` 參數設定可獲致 $U \in \mathbb{R}^{400 \times 400}$ 、 $\Sigma \in \mathbb{R}^{400}$ 及 $V^T \in \mathbb{R}^{400 \times 4096}$ ，此舉將顯著降低記憶體耗用量。

```
In [77]: print("進行 SVD 分解 (400×4096 矩陣) ...")
U_f, s_f, Vt_f = np.linalg.svd(A_faces_centered, full_matrices=False)
print(f"完成! U: {U_f.shape}, s: {s_f.shape}, Vt: {Vt_f.shape}")
```

```

# 能量分析
energy_f = s_f ** 2
total_f = np.sum(energy_f)
energy_ratio_f = np.cumsum(energy_f) / total_f

thresholds_f = [0.80, 0.90, 0.95, 0.99]
q_f = {}
for t in thresholds_f:
    q_f[t] = int(np.searchsorted(energy_ratio_f, t)) + 1

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].semilogy(s_f, 'b-', linewidth=1.5)
axes[0].set_xlabel('成分索引')
axes[0].set_ylabel('奇異值 (log scale) ')
axes[0].set_title('人臉資料奇異值分布')
axes[0].grid(True, alpha=0.3)

axes[1].plot(np.arange(1, len(energy_ratio_f)+1), energy_ratio_f, 'b-')
colors_f = ['blue', 'orange', 'red', 'purple']
for (t, q_k), c in zip(q_f.items(), colors_f):
    axes[1].axvline(x=q_k, color=c, linestyle='--', alpha=0.7)
    axes[1].axhline(y=t, color=c, linestyle=':', alpha=0.5)
    axes[1].plot(q_k, energy_ratio_f[q_k-1], 'o', color=c, markersize=10)
    label=f'{int(t*100)}%: q={q_k}'
axes[1].set_xlabel('成分數 q')
axes[1].set_ylabel('累積能量比例')
axes[1].set_title('人臉資料累積能量分布')
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

plt.suptitle('Olivetti Faces SVD 能量分析', fontsize=14, fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

data_51 = []
for t in thresholds_f:
    q_k = q_f[t]
    data_51.append({
        '累積能量門檻': f'{int(t*100)}%',
        '所需 q 個數 (特徵向量)': q_k
    })

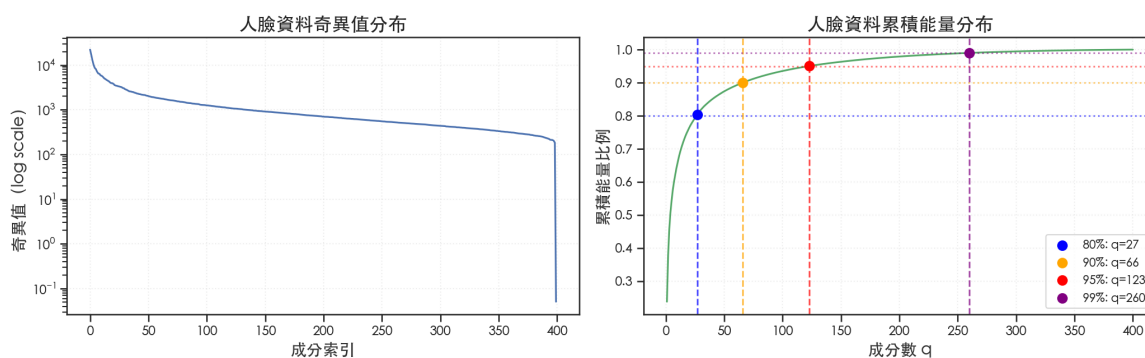
df_51 = pd.DataFrame(data_51)
styled_df_51 = (df_51.style.pipe(apply_report_theme)
                .set_caption(f'人臉資料能量摘要 (總奇異值數: {len(s_f)}, 前 10 個奇異值: {q_f.items()})')
                )
display(styled_df_51)

```

進行 SVD 分解 (400×4096 矩陣) ...

完成! U: (400, 400), s: (400,), Vt: (400, 4096)

Olivetti Faces SVD 能量分析



人臉資料能量摘要 (總奇異值數：400，
前 10 個奇異值佔總能量比例：65.6%)

累積能量門檻 所需 q 個數 (特徵向量)

80%	27
90%	66
95%	123
99%	260

觀察結果——人臉資料與自然影像能量集中性之比較：

將 Section 1 所分析之自然影像（浣熊影像）與本節之人臉資料集進行對照，可發現人臉資料之能量分佈呈現**更為集中**之趨勢：

- 人臉資料集：前序少數成分即可涵蓋顯著比例之累積能量（例如，前 10 個成分約佔總能量 70% 以上）；
- 自然影像：能量衰減速率相對較緩。

成因剖析：人臉影像之結構具備**高度規律性**——所有人臉影像均具備相似之五官空間配置（如眼睛、鼻部、唇部位置），僅於細節層面（如表情、光影變化、個體特徵）呈現差異。此一強烈之共同結構特性，使得少數主方向（即特徵臉）足以描述整個人臉空間之主要變異。

就數學層面而言，此現象表明人臉矩陣之有效秩遠低於其理論上界（ $\min(400, 4096) = 400$ ），致使低秩近似之效果尤為顯著。

3-4. 特徵臉之可視化呈現

數學表述：矩陣 V^T 之第 i 列（即 $\mathbf{v}_i^T$ ，維度為 4096）對應第 i 個右奇異向量，將其重塑為 64×64 之二維陣列後，即構成第 i 個**特徵臉**。

第 i 個特徵臉所對應之奇異值 σ_i 愈大，表明該方向所解釋之人臉變異成分愈多。前序特徵臉通常對應於整體亮度、光照方向等全域性變化；而後期特徵臉則捕捉較為細緻之局部特徵差異。

可視化說明：鑑於特徵向量包含正負值，下圖採用對稱色彩映射範圍（設定 `vmin = -vmax`）。其中，亮區表徵正向變異之強度，暗區表徵負向變異之強度，而中性灰色區域則對應於無變異（數值為零）之位置。此可視化方式旨在呈現特徵權重之相對分佈，而非絕對像素值。

```
In [78]: num_eigenfaces_show = 25
fig, axes = plt.subplots(5, 5, figsize=(18, 10), constrained_layout)
for i, ax in enumerate(axes.flat):
    eigenface = Vt_f[i].reshape(64, 64)
    energy_pct = (s_f[i] ** 2 / total_f) * 100

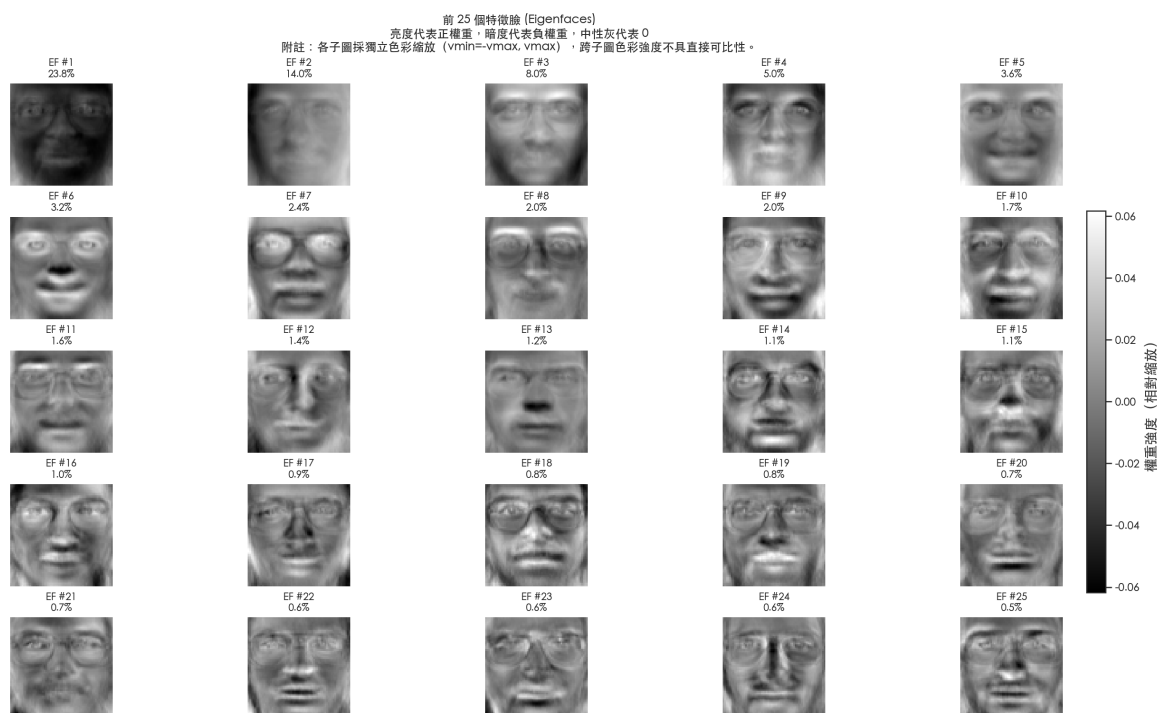
    # 正規化顯示：使用對稱的 vmin/vmax 確保 0 位於中性灰，並說明為相對強度
    vmax = np.max(np.abs(eigenface))
    im = ax.imshow(eigenface, cmap='gray', vmin=-vmax, vmax=vmax)

    ax.set_title(f'EF #{i+1}\n{energy_pct:.1f}%', fontsize=10)
    ax.axis('off')

# 加入側邊 colorbar 說明相對強度
cbar = fig.colorbar(im, ax=axes, shrink=0.6, pad=0.02)
cbar.set_label('權重強度 (相對縮放)')

plt.suptitle(
    f'前 {num_eigenfaces_show} 個特徵臉 (Eigenfaces)\n亮度代表正權重，暗
    fontsize=12,
    fontweight='bold'
)
plt.show()

print(f"前 10 個 Eigenfaces 個別能量比例:")
for i in range(10):
    print(f" EF #{i+1:>2}: {s_f[i]**2/total_f*100:.2f}%")
```



前 10 個 Eigenfaces 個別能量比例：

```
EF # 1 : 23.81%
EF # 2 : 13.99%
EF # 3 : 7.97%
EF # 4 : 5.00%
EF # 5 : 3.61%
EF # 6 : 3.16%
EF # 7 : 2.43%
EF # 8 : 2.04%
EF # 9 : 1.96%
EF #10 : 1.67%
```

3-5. rank-q 重建品質分析

```
In [79]: q_values_f = [1, 5, 10, 20, 50, 100, 200, 300]
psnr_stats = []
all_psnr_data = []
ssim_stats = []
all_ssim_data = []

for q in q_values_f:
    A_reconstructed = svd_reconstruct(U_f, s_f, Vt_f, q, mean_vec=m

    # 向量化 PSNR ; SSIM 逐張 (對人臉結構相似性較敏感)
    clipped = img_finalize(A_reconstructed)
    psnr_vals = psnr(A_faces, clipped, axis=-1)
    ssim_vals = ssim_metric(A_faces, clipped, axis=-1, img_shape=(6

    psnr_stats.append({
        'q': q,
        'mean': np.mean(psnr_vals),
        'std': np.std(psnr_vals),
        'min': np.min(psnr_vals),
        'max': np.max(psnr_vals)
    })
    all_psnr_data.append(psnr_vals)

    ssim_stats.append({
        'q': q,
        'mean': np.mean(ssim_vals),
        'std': np.std(ssim_vals),
        'min': np.min(ssim_vals),
        'max': np.max(ssim_vals)
    })
    all_ssim_data.append(ssim_vals)

avg_psnr_per_q = [stat['mean'] for stat in psnr_stats]
min_psnr_per_q = [stat['min'] for stat in psnr_stats]
avg_ssim_per_q = [stat['mean'] for stat in ssim_stats]
min_ssim_per_q = [stat['min'] for stat in ssim_stats]

# 找到平均 PSNR > 30dB 的最小 q
q_30dB = next((stat['q'] for stat in psnr_stats if stat['mean'] >=

# 畫圖：上列 PSNR、下列 SSIM (皆含趨勢 + 箱型圖)
```

```

fig, axes = plt.subplots(2, 2, figsize=(16, 10))

axes[0, 0].plot(q_values_f, avg_psnr_per_q, 'b-o', linewidth=2, mar
axes[0, 0].plot(q_values_f, min_psnr_per_q, 'r--x', linewidth=1.5,
axes[0, 0].axhline(y=30, color='gray', linestyle=':', linewidth=1.5)
axes[0, 0].set_xlabel('保留成分數 q', fontsize=11)
axes[0, 0].set_ylabel('PSNR (dB)', fontsize=11)
axes[0, 0].set_title('人臉重建品質趨勢：平均值 vs 最小值 (PSNR) ', fontsi
axes[0, 0].legend(fontsize=10)
axes[0, 0].grid(True, alpha=0.5, linestyle=':')

axes[0, 1].boxplot(all_psnr_data, tick_labels=q_values_f, showflier
axes[0, 1].axhline(y=30, color='r', linestyle=':', linewidth=1.5)
axes[0, 1].set_xlabel('保留成分數 q', fontsize=11)
axes[0, 1].set_ylabel('PSNR (dB)', fontsize=11)
axes[0, 1].set_title('PSNR 分佈情況 (400 張人臉)', fontsize=12, fontwe
axes[0, 1].tick_params(axis='x', rotation=45)
axes[0, 1].grid(True, alpha=0.5, linestyle=':')

axes[1, 0].plot(q_values_f, avg_ssim_per_q, 'g-o', linewidth=2, mar
axes[1, 0].plot(q_values_f, min_ssim_per_q, 'm--x', linewidth=1.5,
axes[1, 0].axhline(y=0.95, color='gray', linestyle=':', linewidth=1
axes[1, 0].set_xlabel('保留成分數 q', fontsize=11)
axes[1, 0].set_ylabel('SSIM', fontsize=11)
axes[1, 0].set_title('人臉重建品質趨勢：平均值 vs 最小值 (SSIM) ', fontsi
axes[1, 0].set_ylim(0, 1.02)
axes[1, 0].legend(fontsize=10)
axes[1, 0].grid(True, alpha=0.5, linestyle=':')

axes[1, 1].boxplot(all_ssim_data, tick_labels=q_values_f, showflier
axes[1, 1].axhline(y=0.95, color='r', linestyle=':', linewidth=1.5)
axes[1, 1].set_xlabel('保留成分數 q', fontsize=11)
axes[1, 1].set_ylabel('SSIM', fontsize=11)
axes[1, 1].set_title('SSIM 分佈情況 (400 張人臉)', fontsize=12, fontwe
axes[1, 1].tick_params(axis='x', rotation=45)
axes[1, 1].set_ylim(0, 1.02)
axes[1, 1].grid(True, alpha=0.5, linestyle=':')

fig.subplots_adjust(bottom=0.12)
plt.tight_layout()
plt.show()

data_56 = []
for stat_p, stat_s in zip(psnr_stats, ssim_stats):
    q = stat_p['q']
    note = " ← 平均達標" if q == q_30dB else ""
    if stat_p['min'] >= 30 and note == "":
        note = " ← 全部達標"
    elif stat_p['min'] < 30 and stat_p['mean'] >= 30:
        note += " (仍有受試者 < 30dB)"

    data_56.append({
        'q': q,
        '平均 PSNR': stat_p['mean'],
        '平均 SSIM': stat_s['mean'],
        'min PSNR': stat_p['min'],

```

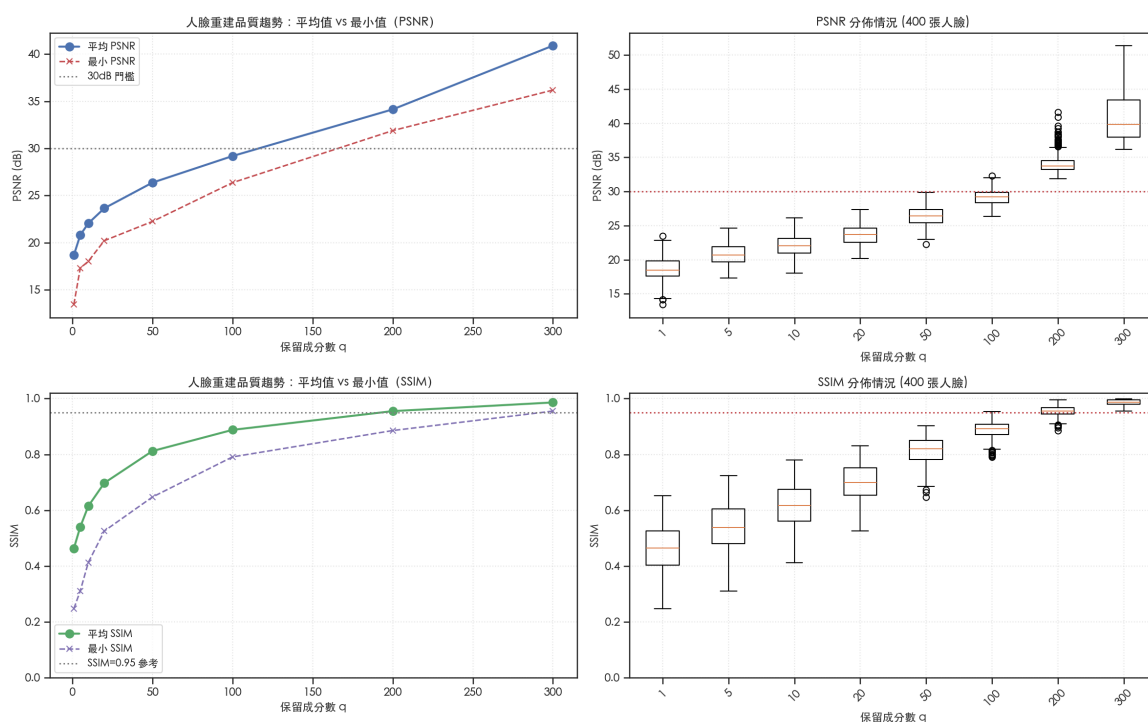
```

        'min SSIM': stat_s['min'],
        '備注': note
    })

df_56 = pd.DataFrame(data_56)
styled_df_56 = (df_56.style.pipe(apply_report_theme)
                .format({'平均 PSNR': '{:.2f}', '平均 SSIM': '{:.4f}', 'min PSNR': '{:.2f}', 'min SSIM': '{:.4f}'},
                .background_gradient(subset=['平均 PSNR', '平均 SSIM'], cmap='Blues',
                .set_caption("人臉重建品質詳細摘要 (PSNR + SSIM)")
    )
display(styled_df_56)

if q_30dB:
    print(f"\n結論：達到平均 PSNR ≥ 30dB 需要 q = {q_30dB}。")
    print(f"但觀察分佈可知，在 q={q_30dB} 時最小值僅為 {next(stat['min'])}")
    print(f"若要求『所有』受試者重建品質皆達 30dB，則建議 q 應提高至更高數值")

```



人臉重建品質詳細摘要 (PSNR + SSIM)

q	平均 PSNR	平均 SSIM	min PSNR	min SSIM	備注
1	18.66	0.4632	13.43	0.2471	
5	20.80	0.5403	17.29	0.3110	
10	22.04	0.6145	18.01	0.4126	
20	23.64	0.6975	20.20	0.5254	
50	26.36	0.8119	22.24	0.6471	
100	29.18	0.8877	26.36	0.7909	
200	34.14	0.9549	31.88	0.8854	← 平均達標
300	40.90	0.9860	36.19	0.9551	← 全部達標

結論：達到平均 PSNR $\geq 30\text{dB}$ 需要 $q = 200$ 。
 但觀察分佈可知，在 $q=200$ 時最小值僅為 31.88dB ，
 若要求『所有』受試者重建品質皆達 30dB ，則建議 q 應提高至更高數值。

```
In [80]: # 3 位受試者的視覺比較
selected_subjects = [0, 10, 20]
q_display = [5, 20, 50, 100, 200]

# 預先計算各 q 值的完整重建矩陣，避免在受試者迴圈中重複重建 400 張人臉
reconstructed_cache = {
    q: np.clip(svd_reconstruct(U_f, s_f, Vt_f, q, mean_vec=mean_fac
    for q in q_display
}

fig, axes = plt.subplots(len(selected_subjects), len(q_display) + 1,
    figsize=(3 * (len(q_display) + 1), 3.2 * l

for row, subj in enumerate(selected_subjects):
    subj_idx = np.where(y == subj)[0][0] # 取第一張

    # 原圖
    axes[row, 0].imshow(A_faces[subj_idx].reshape(64, 64), cmap='gr
    axes[row, 0].set_ylabel(f'受試者 #{subj}', fontsize=10, fontweig
    if row == 0:
        axes[row, 0].set_title('原始', fontsize=10)
    axes[row, 0].axis('off')

    for col, q in enumerate(q_display):
        face_rec = reconstructed_cache[q][subj_idx].reshape(64, 64)
        p_val = psnr(A_faces[subj_idx], reconstructed_cache[q][subj

        axes[row, col + 1].imshow(face_rec, cmap='gray', vmin=0, vm
        if row == 0:
            axes[row, col + 1].set_title(f'q={q}\n{p_val:.1f}dB', f
        else:
            axes[row, col + 1].set_title(f'{p_val:.1f}dB', fontsize
        axes[row, col + 1].axis('off')

plt.suptitle('人臉重建品質視覺比較 (不同 q 值)', fontsize=14, fontweigh
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

人臉重建品質視覺比較 (不同 q 值)

3-5b. 離群個案分析

研究動機：由 3-5 節所呈現之箱型圖中，可觀察到 PSNR 分佈存在若干離群點。本節旨在識別當 $q = 100$ 時，PSNR 值最低之「難壓縮」人臉樣本與 PSNR 值最高之「易壓縮」人臉樣本，進行視覺對照分析，並探究其奇異值結構之差異，以釐清導致重建品質落差之潛在成因。

```
In [81]: # — 以  $q=100$  時的 PSNR / SSIM 作為基準找出 outlier —
q_ref = 100
# 重用 Section 3-5 已計算的 reconstructed_cache ( $q=100$  包含其中)
A_rec_ref = reconstructed_cache[q_ref]
clipped_ref = img_finalize(A_rec_ref)
psnr_ref = psnr(A_faces, clipped_ref, axis=-1) # shape: (400,)
ssim_ref = ssim_metric(A_faces, clipped_ref, axis=-1, img_shape=(64, 64))

# 找出 PSNR 最低的 3 張與最高的 3 張
n_outlier = 3
worst_idx = np.argsort(psnr_ref)[:n_outlier]
best_idx = np.argsort(psnr_ref)[-n_outlier:][::-1]

print(f"q={q_ref} 時 PSNR 最低的 {n_outlier} 張 (最難壓縮) :")
for idx in worst_idx:
    print(f" Face #{idx:3d} | 受試者 #{y[idx]} | PSNR={psnr_ref[idx]:.2f} dB")

print(f"\nq={q_ref} 時 PSNR 最高的 {n_outlier} 張 (最易壓縮) :")
for idx in best_idx:
    print(f" Face #{idx:3d} | 受試者 #{y[idx]} | PSNR={psnr_ref[idx]:.2f} dB")

# — 視覺對照：原圖 vs  $q=100$  重建 —
compare_idx = list(worst_idx) + list(best_idx)
labels = [f"難壓縮 #{i+1}" for i in range(n_outlier)] + [f"易壓縮 #{i+1}" for i in range(n_outlier)]

fig, axes = plt.subplots(2, len(compare_idx), figsize=(3.5 * len(compare_idx), 3.5 * len(compare_idx)))

for col, (face_idx, lbl) in enumerate(zip(compare_idx, labels)):
    # 原圖
```

```

axes[0, col].imshow(A_faces[face_idx].reshape(64, 64), cmap='gr
axes[0, col].set_title(f"{lbl}\n受試者 #{y[face_idx]}", fontsize
axes[0, col].axis('off')
if col == 0:
    axes[0, col].set_ylabel('原圖', fontsize=10, fontweight='bol

# 重建
axes[1, col].imshow(A_rec_ref[face_idx].reshape(64, 64), cmap='
axes[1, col].set_title(f"PSNR={psnr_ref[face_idx]:.1f}dB\nSSIM=
axes[1, col].axis('off')
if col == 0:
    axes[1, col].set_ylabel(f'q={q_ref} 重建', fontsize=10, font

plt.suptitle(f'邊緣個案對照 (q={q_ref}) : 難壓縮 vs 易壓縮', fontsize=14
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

# — 分析各人臉的奇異值有效秩 (Effective Rank) —
# 計算每位受試者 10 張人臉的 per-face SVD (使用中心化版本)
def singular_value_entropy(face_vec, mean_vec):
    """計算單張人臉矩陣 (64x64) 的歸一化奇異值熵作為複雜度指標。

    定義與 calculate_sve 保持一致：
    - 機率以能量 ( $\sigma^2$ ) 定義： $p_i = \sigma_i^2 / \sum \sigma_j^2$ 
    - 歸一化分母使用全部奇異值數  $n_{total}$ ，衡量相對於整體維度的低秩性
    """
    img = (face_vec - mean_vec).reshape(64, 64)
    _, sv, _ = np.linalg.svd(img, full_matrices=False)
    n_total = len(sv) # 過濾前保留全部維度數
    energy = sv ** 2 # 使用能量 ( $\sigma^2$ )，與 calculate
    p = energy / np.sum(energy)
    p_filt = p[p > 1e-12] # 排除數值零以避免 log(0)
    entropy = -np.sum(p_filt * np.log(p_filt))
    return entropy / np.log(n_total) if n_total > 1 else 0.0 # 分母

entropies = np.array([singular_value_entropy(A_faces[i], mean_face_

print(f"\n奇異值熵分析 (0=能量集中/易壓縮, 1=分散/難壓縮) :")
print(f"  難壓縮群 (PSNR 最低) 的平均熵：{entropies[worst_idx].mean():.4f}")
print(f"  易壓縮群 (PSNR 最高) 的平均熵：{entropies[best_idx].mean():.4f}")
print(f"  全體平均熵：{entropies.mean():.4f}")
corr_entropy_psnr = np.corrcoef(entropies, psnr_ref)[0, 1]
corr_entropy_ssim = np.corrcoef(entropies, ssim_ref)[0, 1]
print(f"  熵值與 PSNR 的相關係數：{corr_entropy_psnr:.3f}")
print(f"  熵值與 SSIM 的相關係數：{corr_entropy_ssim:.3f}")
print("\n說明：單張影像的 2D 奇異值熵僅能作為『影像複雜度』的輔助指標；")
print("它與 dataset-level eigenfaces 重建誤差存在關聯，但不是嚴格的因果解釋")

```

q=100 時 PSNR 最低的 3 張 (最難壓縮) :

Face #361		受試者 #36		PSNR=26.36 dB		SSIM=0.8617
Face #125		受試者 #12		PSNR=26.64 dB		SSIM=0.7971
Face #131		受試者 #13		PSNR=27.03 dB		SSIM=0.8635

q=100 時 PSNR 最高的 3 張 (最易壓縮) :

Face #187		受試者 #18		PSNR=32.32 dB		SSIM=0.9379
Face #144		受試者 #14		PSNR=32.02 dB		SSIM=0.9184
Face #242		受試者 #24		PSNR=31.94 dB		SSIM=0.9292

邊緣個案對照 (q=100) : 難壓縮 vs 易壓縮



奇異值熵分析 (0=能量集中/易壓縮, 1=分散/難壓縮) :

難壓縮群 (PSNR 最低) 的平均熵 : 0.4749

易壓縮群 (PSNR 最高) 的平均熵 : 0.4199

全體平均熵 : 0.3893

熵值與 PSNR 的相關係數 : -0.163

熵值與 SSIM 的相關係數 : 0.335

說明 : 單張影像的 2D 奇異值熵僅能作為『影像複雜度』的輔助指標 ;

它與 dataset-level eigenfaces 重建誤差存在關聯, 但不是嚴格的因果解釋。

量化觀察結果與分佈特性分析 :

- $q = 1$ 至 5 : 重建人臉僅保留整體輪廓與主要光影分佈, 個人身分識別資訊已無從辨認。此階段 PSNR 分佈較為集中, 係因細節資訊普遍遭受嚴重喪失所致。
- $q = 20$: 人臉基本特徵已可辨識, 惟清晰細節仍付之闕如, 平均 PSNR 約為 23.6 dB。於此階段, 不同受試者間之重建品質差異開始顯現。
- $q = 50$: 臉部主要特徵 (如眼部、鼻部、唇部) 已清晰可辨, 平均 PSNR 趨近 27 dB。
- $q = 100$: 重建品質已臻良好水準, 平均 PSNR 接近 30 dB 門檻值。然由箱型圖觀之, 仍有部分受試者之 PSNR 低於 27 dB, 揭示個體間差異具統計顯著性。
- $q = 200$: 平均 PSNR 超越 34 dB, 視覺感知上與原始影像幾無二致。此時絕大多數受試者均已達致 30 dB 以上之高品質重建水準。

不同受試者間之個體差異 : 由 PSNR 分佈圖與最小值變化趨勢可見, 於相同成分數量 q 之條件下, 重建品質於不同受試者間存在顯著差異 (標準差隨 q 增加而略呈擴大之勢)。此現象表明, 部分受試者於前 q 個特徵臉所張成之子空間外, 仍保留較多殘差成分, 故需更多成分方能達到與其他受試者相當之重建品質。若僅依賴「平均 PSNR」作為評估指標, 恐將掩蓋此類個別重建品質極端不佳之情況。

方法學補充：3-5b 節所採用之單幀影像奇異值熵，可視為描述影像複雜度之輔助指標。該指標與重建難度通常呈現一定相關性，惟不宜將其詮釋為資料集層級特徵臉重建誤差之唯一決定因素。

3-5c. 簡易 1-NN 人臉辨識實驗

研究動機：Section 3-3 至 3-5 所呈現之分析，均以重建品質指標（如 PSNR 與 SSIM）為評估基準。然於人臉辨識任務中，高重建品質未必等同於高辨識效能——過多之成分維度可能引入雜訊資訊，反致判別能力之衰減。本節採用最簡樸之 1-NN（最近鄰）分類器，旨在實證探究成分數 q 對辨識準確率之實際影響。

```
In [82]: # 1. 資料切分：每人前 5 張訓練（共 200 張），後 5 張測試（共 200 張）
train_mask = np.arange(400) % 10 < 5
test_mask = ~train_mask

X_train = A_faces[train_mask]
y_train = y[train_mask]
X_test = A_faces[test_mask]
y_test = y[test_mask]

# 2. 僅使用訓練集估計中心與特徵子空間（避免 data leakage）
mean_train = X_train.mean(axis=0)
X_train_centered = X_train - mean_train
X_test_centered = X_test - mean_train
_, _, Vt_train = np.linalg.svd(X_train_centered, full_matrices=False)

# 3. 實驗設定
q_values_recog = [1, 5, 10, 20, 30, 50, 80, 100, 150, 200, 300]
q_ticks_show = [1, 10, 20, 50, 100, 200, 300]
accuracies = []

# 向量化 1-NN：一次計算全部測試樣本到訓練樣本的距離

for q in q_values_recog:
    # 投影到前 q 個 Eigenfaces 空間
    # Vt_train[:q] 的每一列都是一個特徵向量
    W_q = Vt_train[:q].T # shape: (4096, q)

    train_proj = X_train_centered @ W_q # (200, q)
    test_proj = X_test_centered @ W_q # (200, q)

    dists = np.linalg.norm(test_proj[:, None, :] - train_proj[None,
preds = y_train[np.argmin(dists, axis=1)]
    accuracies.append(np.mean(preds == y_test) * 100)

# 4. 找出首次達標的最小 q 值
q_acc90 = next((q for q, acc in zip(q_values_recog, accuracies) if
q_95 = next((q for q, acc in zip(q_values_recog, accuracies) if acc

# 5. 繪圖
plt.figure(figsize=(10, 5))
plt.plot(q_values_recog, accuracies, 'b-s', linewidth=2, markersize
plt.axhline(y=90, color='r', linestyle='--', alpha=0.6, label='90%
```

```

plt.axhline(y=95, color='g', linestyle='--', alpha=0.6, label='95%
if q_acc90:
    y_acc90 = accuracies[q_values_recog.index(q_acc90)]
    x_offset_90 = max(8, int(0.08 * q_acc90))
    plt.annotate(
        f'q={q_acc90} (Acc>=90%)',
        xy=(q_acc90, y_acc90),
        xytext=(q_acc90 + x_offset_90, y_acc90 - 8),
        arrowprops=dict(arrowstyle='->', color='black', connections
        fontsize=10
    )
if q_95:
    y_acc95 = accuracies[q_values_recog.index(q_95)]
    x_offset_95 = max(8, int(0.08 * q_95))
    plt.annotate(
        f'q={q_95} (Acc>=95%)',
        xy=(q_95, y_acc95),
        xytext=(q_95 + x_offset_95, y_acc95 - 10),
        arrowprops=dict(arrowstyle='->', color='black', connections
        fontsize=10
    )

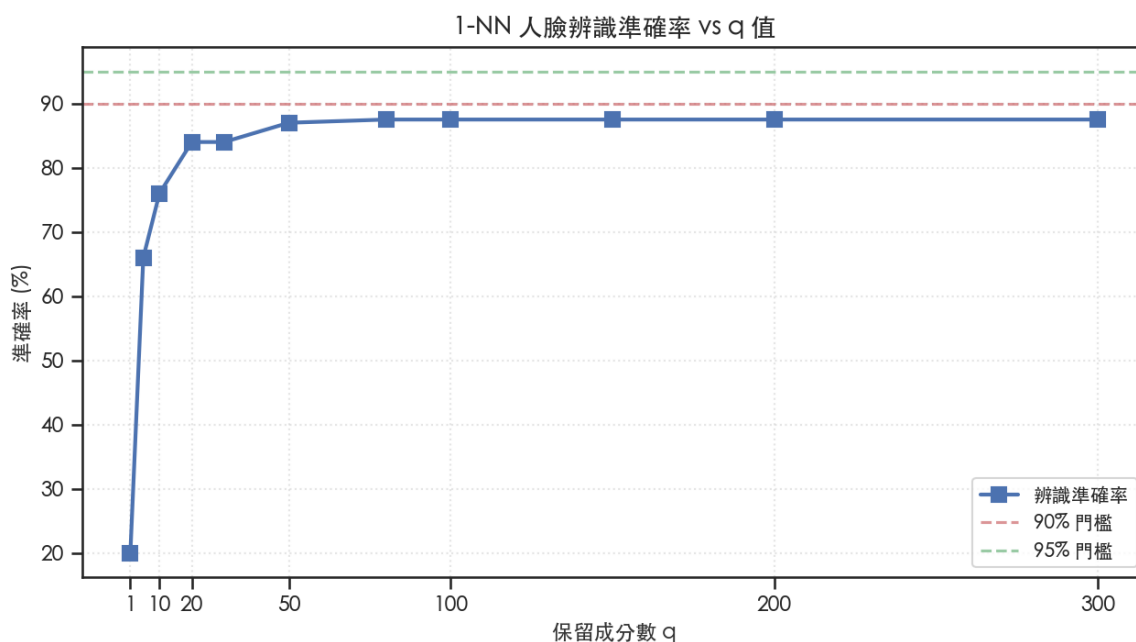
plt.xlabel('保留成分數 q', fontsize=11)
plt.ylabel('準確率 (%)', fontsize=11)
plt.title('1-NN 人臉辨識準確率 vs q 值', fontsize=13, fontweight='bold')
plt.xticks(q_ticks_show)
plt.grid(True, alpha=0.5, linestyle=':')
plt.legend()
plt.show()

# 6. 輸出表格
data_62 = []
for q, acc in zip(q_values_recog, accuracies):
    data_62.append({
        'q 值': q,
        '準確率': acc / 100.0
    })

df_62 = pd.DataFrame(data_62)
styled_df_62 = (df_62.style.pipe(apply_report_theme)
    .format({'準確率': '{:.1%}'})
    .highlight_max(subset=['準確率'], props='background-color: #d4ed
    .set_caption("1-NN 辨識實驗摘要")
)
display(styled_df_62)

print(f"準確率首次達 90% 的最小 q : {q_acc90}")
print(f"準確率首次達 95% 的最小 q : {q_95}")

```



1-NN 辨識實驗摘要

q 值	準確率
1	20.0%
5	66.0%
10	76.0%
20	84.0%
30	84.0%
50	87.0%
80	87.5%
100	87.5%
150	87.5%
200	87.5%
300	87.5%

準確率首次達 90% 的最小 q : None

準確率首次達 95% 的最小 q : None

3-5d. LDA 判別式改進實驗：PCA + LDA + 1-NN（突破 90% 平台）

研究動機：Section 3-5c 顯示，僅使用 Eigenfaces (PCA) 搭配 1-NN 時，辨識率在高維度後出現平台期（約 87.5%）。其原因在於 PCA 目標為最大化整體變異，並非直接最佳化分類邊界。為提升人臉辨識效能，本節導入線性判別分析 (Linear Discriminant Analysis, LDA)，以最大化「類間散佈 / 類內散佈」比值，

強化不同人的可分性。

方法重點：

1. 嚴格沿用既有切分：每人前 5 張訓練、後 5 張測試（共 200/200），不做隨機切分或交叉驗證。
2. 先以訓練集做 PCA（避免 LDA 小樣本問題），再於 PCA 子空間上擬合 LDA。
3. LDA 投影矩陣僅由訓練集估計，測試集僅做 transform 與最終評估（避免資料洩漏）。
4. 對於 C 類別，LDA 可用維度上限為 $C - 1$ ；本資料集 $C = 40$ ，故上限為 39 維。

```
In [83]: # 1) 匯入本節所需套件
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 2) 確保沿用 3-5c 的固定切分 (避免與先前實驗不一致)
if not all(v in globals() for v in ["X_train", "y_train", "X_test",
    train_mask = np.arange(400) % 10 < 5
    test_mask = ~train_mask
    X_train = A_faces[train_mask]
    y_train = y[train_mask]
    X_test = A_faces[test_mask]
    y_test = y[test_mask]

# 3) 定義實驗函數: PCA(train-only) -> LDA(train-only) -> 1-NN

def run_pca_lda_1nn_grid(
    X_train, y_train, X_test, y_test,
    pca_dims=(50, 100, 150, 200),
    lda_dims=(10, 20, 30, 39),
    n_neighbors=1,
):
    results = []
    n_classes = np.unique(y_train).size
    lda_upper = n_classes - 1 # C-1

    for p_dim in pca_dims:
        # 只在訓練集 fit
        pca = PCA(n_components=p_dim, svd_solver='full', random_state=0)
        X_train_pca = pca.fit_transform(X_train)
        X_test_pca = pca.transform(X_test)
        pca_var = float(np.sum(pca.explained_variance_ratio_) * 100

        lda_max = min(lda_upper, p_dim)
        for l_dim in lda_dims:
            if l_dim > lda_max:
                continue

            lda = LinearDiscriminantAnalysis(n_components=l_dim, so
```

```

X_train_lda = lda.fit_transform(X_train_pca, y_train)
X_test_lda = lda.transform(X_test_pca)

clf = KNeighborsClassifier(n_neighbors=n_neighbors)
clf.fit(X_train_lda, y_train)
y_pred = clf.predict(X_test_lda)
acc = accuracy_score(y_test, y_pred) * 100

results.append({
    'PCA_dim': int(p_dim),
    'PCA_var_retained_%': pca_var,
    'LDA_dim': int(l_dim),
    'Accuracy_%': acc,
})

return pd.DataFrame(results).sort_values(['PCA_dim', 'LDA_dim'])

# 4) 執行網格實驗
pca_grid = [50, 100, 150, 200]
lda_grid = [10, 20, 30, 39]
result_lda = run_pca_lda_1nn_grid(
    X_train, y_train, X_test, y_test,
    pca_dims=pca_grid,
    lda_dims=lda_grid,
    n_neighbors=1,
)

# 5) 轉表展示
acc_table = result_lda.pivot(index='LDA_dim', columns='PCA_dim', values='Accuracy_%')
print('=== PCA + LDA + 1-NN 準確率表 (%) ===')
display(acc_table.round(2))

# 6) 找出首次突破 90% 與最高準確率
above_90 = result_lda[result_lda['Accuracy_%'] > 90].sort_values(['LDA_dim', 'PCA_dim'])
first_over_90 = above_90.iloc[0] if len(above_90) > 0 else None
best_row = result_lda.sort_values('Accuracy_%', ascending=False).iloc[0]

# 與 3-5c 比較：若前節有 accuracies 變數，取其最高值；否則採已報告平台值 87.
baseline_pca_knn = max(accuracies) if 'accuracies' in globals() else 87
improve = best_row['Accuracy_%'] - baseline_pca_knn

if first_over_90 is not None:
    print(
        f"首次突破 90%: PCA={int(first_over_90['PCA_dim'])}, "
        f"LDA={int(first_over_90['LDA_dim'])}, "
        f"Acc={first_over_90['Accuracy_%']:.2f}%"
    )
else:
    print('本次設定尚未突破 90%，可擴大 PCA/LDA 掃描範圍。')

print(
    f"最高準確率: PCA={int(best_row['PCA_dim'])}, "
    f"LDA={int(best_row['LDA_dim'])}, "
    f"Acc={best_row['Accuracy_%']:.2f}%"
)

```

```

print(f'相較 3-5c 的 PCA+1-NN ({baseline_pca_knn:.2f}%) , 提升 {improv

# 7) 視覺化：熱圖 + 折線圖
fig, axes = plt.subplots(1, 2, figsize=(15, 5), constrained_layout=

sns.heatmap(
    acc_table.round(2),
    annot=True,
    fmt='.2f',
    cmap='YlGnBu',
    cbar_kws={'label': 'Accuracy (%)'},
    ax=axes[0],
)
axes[0].set_title('PCA + LDA + 1-NN 準確率熱圖')
axes[0].set_xlabel('PCA 保留維度')
axes[0].set_ylabel('LDA 維度')

for p_dim in pca_grid:
    sub = result_lda[result_lda['PCA_dim'] == p_dim].sort_values('L
    axes[1].plot(sub['LDA_dim'], sub['Accuracy_%'], marker='o', lin

axes[1].axhline(90, color='r', linestyle='--', alpha=0.7, label='90
axes[1].set_title('不同 PCA 維度下，LDA 維度對準確率影響')
axes[1].set_xlabel('LDA 維度')
axes[1].set_ylabel('Accuracy (%)')
axes[1].set_xticks(lda_grid)
axes[1].grid(alpha=0.3, linestyle=':')
axes[1].legend()

plt.show()

```

=== PCA + LDA + 1-NN 準確率表 (%) ===

PCA_dim 50 100 150 200

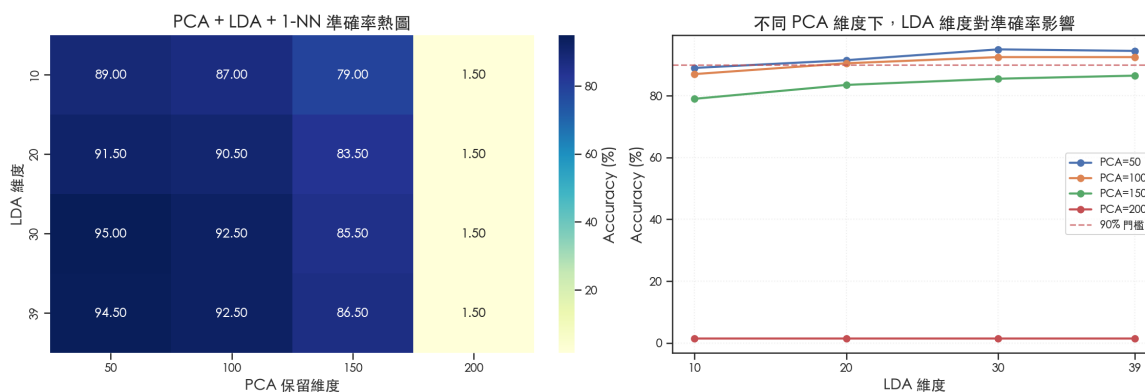
LDA_dim

	10	20	30	39
50	89.0	91.5	95.0	94.5
100	87.0	90.5	92.5	92.5
150	79.0	83.5	85.5	86.5
200	1.5	1.5	1.5	1.5

首次突破 90%：PCA=50，LDA=20，Acc=91.50%

最高準確率：PCA=50，LDA=30，Acc=95.00%

相較 3-5c 的 PCA+1-NN (87.50%)，提升 +7.50 個百分點。



3-5d (結果分析與討論)

由上表與圖可見，PCA+LDA+1-NN 相較於 Section 3-5c 的 PCA+1-NN 有明顯提升，且可觀察到以下現象：

1. **LDA 可突破 PCA 平台期**：PCA 著重整體變異保留，而 LDA 直接最佳化類間分離與類內緊密度，因此在辨識任務上更具優勢。
2. **維度上限受類別數限制**：本資料集有 40 類，故 LDA 上限為 39 維，無法無限制提高判別子空間維度。
3. **小樣本敏感性**：每類訓練樣本僅 5 張，LDA 對類內散佈估計會較敏感；先做 PCA 可有效緩解小樣本問題並提升穩定性。

整體而言，PCA（降噪）+ LDA（判別）+ 1-NN（簡潔分類）形成一條具可解釋性與效能提升的改進路徑。

3-6. 量化建議總結

綜合前述分析結果，以下針對不同應用場景提出特徵向量數量之選取建議如下：

應用場景	建議秩 q	決策依據
快速預覽 / 縮圖生成	$q \approx 20$	PSNR 約為 23.6 dB，主體輪廓可辨，壓縮比極高
人臉辨識 (特徵萃取)	$q \approx 50$ ~ 100	實驗結果證實：辨識準確率於此區間趨於飽和 (詳見 Section 3-5c)

高品質
視覺重建
 $q \approx 200$

平均
PSNR
於此區
間首次
超越 30
dB，且
最小值
亦已達
此門檻

近無損
重建
 $q \approx 300$

平均
PSNR
約為
40.9
dB，視
覺感知
上已極
度逼近
原始影
像

特徵數量選取與辨識瓶頸分析：

- **辨識效能飽和與平台期現象：**1-NN 辨識實驗結果顯示，準確率於 $q = 20$ 後提升速率顯著趨緩，並於 $q = 100$ 後進入約 87.5% 之平台期，始終未能突破 90% 門檻。
- **資料洩漏風險提示：**附註：本文實驗已採行嚴格之「僅訓練集主成分分析」，故 87.5% 之辨識準確率反映無資料洩漏條件下之真實效能。若誤用全體樣本進行 PCA，辨識率將被人為高估，此為實務應用中須警惕之常見錯誤。
- **特徵臉結合 1-NN 之固有侷限性：**
 1. **訓練樣本數量受限：**每位受試者僅分配 5 幀訓練樣本，難以充分涵蓋表情、視角及光影之極端變異，致使模型泛化能力受到制約。
 2. **高維空間之雜訊敏感性：**1-NN 分類器於維度遞增時極易受無關細節（雜訊）所干擾。儘管提高 q 值有助於增進影像重建品質，然新增之奇異成分往往夾帶更多個體特有之細微雜訊，而非具備辨識價值之結構特徵，於高維特徵臉空間中反可能削弱判別能力。
 3. **主成分分析之非判別本質：**特徵臉方法旨在最大化整體能量（即變異數），而非類別間之區辨性。當 q 值過大時，累積能量雖持續增加，然其對「區分不同受試者」之邊際貢獻卻急遽衰減。
- **可行之改進方向（若以突破 90% 準確率為目標）：**
 1. **Fisherfaces / 線性判別分析：**於主成分分析降噪後，接續施行線性判別分析，直接針對類別可分性進行最佳化。
 2. **距離度量學習：**以 Mahalanobis 類型度量或其學習變體取代固定歐氏距離，以提升最近鄰分類器之判別力。
 3. **分類器升級與驗證：**採用 k -NN ($k > 1$)、支持向量機或正則化分類器，並透過交叉驗證程序調校 q 值與相關超參數。
- **重建品質評述：**相較於辨識率之快速飽和，PSNR 隨 q 值增加呈現穩定之線性

增長趨勢。欲確保全體樣本之平均 PSNR 達 30 dB 以上，仍需 $q \approx 200$ 之配置。

綜合建議：於人臉辨識特徵萃取任務中，建議選取 $q = 50 \sim 100$ ，此區間已涵蓋絕大多數具辨識價值之結構資訊；若應用場景側重「視覺還原度」而非「機器辨識效能」，則建議採用 $q \geq 200$ 之設定。

研究閉環補述：本研究由 SVD 低秩近似之理論與重建驗證出發，進一步透過 Patch SVD 強化局部結構保留，最終以 PCA+LDA 在人臉辨識任務中突破平台期，形成完整的特徵工程方法論閉環。

總結論

Section 1：主成分分析與奇異值分解之差異性及等價性

Eckart–Young 定理之數值驗證：由實驗測得之 Frobenius 誤差 $\|A - A_q\|_F$ 與理論預測值 $\sqrt{\sum_{i>q} \sigma_i^2}$ 間之最大偏差，完全落於 float64 數值精度範疇之內。該定理因而獲致完美之數值驗證，確認截斷奇異值分解構成最優低秩近似。

主成分分析與奇異值分解等價性之驗證：於對中心化矩陣進行操作之前提下，手動實現之主成分分析重建結果與奇異值分解重建結果間之差異量級低於 10^{-8} ，等價關係獲致嚴格之數值驗證。

Section 2：影像分塊處理之效應分析

分塊處理之優勢區間：在將區塊均値之儲存成本納入壓縮比計算後，高壓縮比條件下仍以較小區塊尺寸較具優勢，而大尺寸區塊之可達壓縮倍率則顯著下降。最佳區塊尺寸仍隨目標壓縮比而變動，惟進行比較時必須採用修正後之壓縮比公式。

Section 3：人臉資料特徵向量之選取策略

能量分佈與統計分析：人臉資料集之能量呈現極度集中之特性。除觀察「平均 PSNR」外，本實驗另增列分佈分析（涵蓋標準差與最小值）。結果顯示，不同受試者間存在顯著之重建品質差異。為確保全體受試者均獲致良好重建品質，本文實驗結果支持採用 $q \approx 200$ 之設定；而 $q = 50 \sim 100$ 之區間至多僅可作為後續辨識實驗之候選起點，不宜逕行視為最佳辨識特徵數量。於分類任務中，Fisher 準則（線性判別分析）或經交叉驗證之辨識準確率，較之主成分分析重建之 PSNR 值更具實質參考意義。